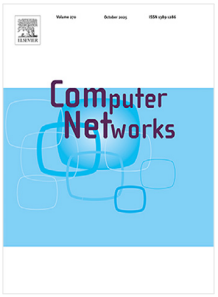
Journal Pre-proof



LITMUS: A lightweight collaborative intrusion detection system for host oriented mimicry attack detection in Industrial Internet of Things networks with dishonest collaborators in the majority

Surja Sanyal, Manas Khatua, Pratik Chattopadhyay

PII: S1389-1286(26)00486-X
DOI: https://doi.org/10.1016/j.comnet.2026.112474
Reference: COMPNW 112474

To appear in: *Computer Networks*

Received date : 16 January 2026
Revised date : 20 May 2026
Accepted date : 7 June 2026

Please cite this article as: S. Sanyal, M. Khatua and P. Chattopadhyay, LITMUS: A lightweight collaborative intrusion detection system for host oriented mimicry attack detection in Industrial Internet of Things networks with dishonest collaborators in the majority, *Computer Networks* (2026), doi: https://doi.org/10.1016/j.comnet.2026.112474.

LITMUS: A Lightweight Collaborative Intrusion Detection System for Host Oriented Mimicry Attack Detection in Industrial Internet of Things Networks with Dishonest Collaborators in the Majority

Surja Sanyal[a,b,*], Manas Khatua[a] and Pratik Chattopadhyay[b]

[a]*Indian Institute of Technology Guwahati, Surjyamukhi Road, Guwahati 781039, Assam, India*
[b]*Indian Institute of Technology (BHU) Varanasi, Lanka Road, Varanasi 221005, Uttar Pradesh, India*

ARTICLE INFO

Keywords:
Industrial Internet of Things (IIoT)
Intrusion Detection System (IDS)
Mimicry Attack
Lightweight IDS Collaboration
Data Privacy
Trust Management
Attack Resiliency

ABSTRACT

The resource-constrained nature and device diversity of the Internet of Things (IoT) necessitate robust security measures to protect IoT networks against attacks. Host-oriented Mimicry Attack (HMA) is an advanced attack that evades traditional intrusion detection systems (IDSs) in IoT by mimicking the affected device's host and network behavior. Hybrid implementations of IDSs combine host-oriented IDS (HIDS) and network-based IDS (NIDS) to detect HMAs effectively. Collaborative IDSs can improve HMA detection accuracy further by exchanging attack information between peer devices. To reduce message exchange overhead, the distributed design of collaborative IDSs is suitable, but it sacrifices HMA detection accuracy. Furthermore, existing approaches do not address data privacy issues in the network and are ineffective for sparsely honest collaborations involving only a few truthful peers. Additionally, the resource consumption in the existing trust-based schemes is high due to neighbor behavior monitoring. Therefore, this work proposes a **L**ightweight d**I**stributed collabora**T**ive IDS for **M**imicry-based intr**U**sion detection in **S**parsely honest Industrial IoT networks (`LITMUS`). `LITMUS` uses a hybrid IDS for HMA detection with high accuracy, a bloom filter-based communication scheme that ensures data privacy and integrity while reducing communication overhead, a frequency-aware collaboration scheme that works in sparsely honest networks, and a lightweight trust management scheme for insider attack protection. Detailed theoretical analysis and extensive experimentation have shown that `LITMUS` markedly outperforms the state of the art in HMA detection in sparsely honest networks, reduces communication size and associated energy consumption, and provides robust trust-based immunity against insider attacks while incurring lower false-positive rates.

1. Introduction

The Internet of Things (IoT) is expanding its reach at an unprecedented rate, with diverse devices permeating people's everyday lives. With this diversity comes an elevated risk of attacks [1, 2, 3] and an urgent need for Intrusion Detection Systems (IDS) [4, 5, 6]. These attacks evolve with time and become increasingly difficult to detect. Host-oriented mimicry attack (HMA) is an advanced attack that affects host devices while evading IDSs by mimicking benign host and network behaviors [7, 8]. The detection techniques applied by IDSs can be classified as signature- or rule-based when they search for known attack footprints [9] and as anomaly-based when they recognize unknown patterns in host or network behavior [10, 11]. IDSs that combine these two approaches are known as hybrid IDSs [12]. IDSs can also be classified by application area as network IDSs (NIDS) [13, 14, 15] and host IDSs (HIDS) [16, 11]. Positional hybrid IDSs combine both approaches [17, 18]. To improve detection effectiveness, machine learning (ML) or deep learning (DL) models are often used as supporting tools in anomaly-based IDSs [19, 20, 21]. Often, such tools help the IDS evolve to detect multi-step stealthy HMA attacks [22, 23]. As the attacker in HMA mimics both host and network behaviors, positional hybrid anomaly-based IDS implementations perform better at HMA detection than pure implementations of either approach [17]. Therefore, a portion of IDS implementation must run on the host, as

*Corresponding author
`surja.sanyal@iitg.ac.in` (S. Sanyal); `surja.sanyal@iitbhu.ac.in` (S. Sanyal); `manaskhatua@iitg.ac.in` (M. Khatua); `pratik.cse@iitbhu.ac.in` (P. Chattopadhyay)
`https://surja-sanyal.github.io/` (S. Sanyal); `https://manaskhatua.github.io/` (M. Khatua); `https://iitbhu.ac.in/dept/cse/people/pratikcse` (P. Chattopadhyay)
ORCID(s): `0000-0002-1308-9433` (S. Sanyal); `0000-0002-8329-2429` (M. Khatua); `0000-0002-5805-6563` (P. Chattopadhyay)

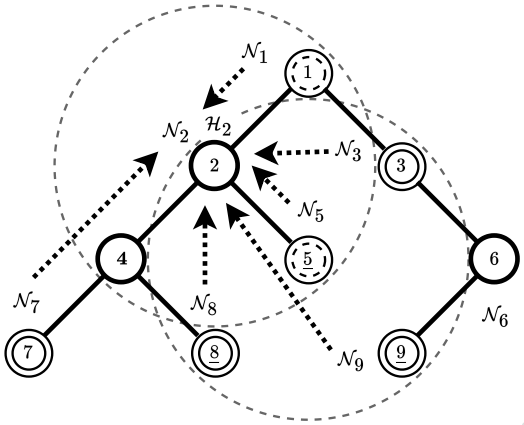


Figure 1: Collusion by the nodes **5**, **8**, and **9** in sparsely honest networks nullifies collaboration benefits. Dashed circles indicate communication ranges.

attack footprints are comparatively more abundant in host behavior than in network traffic. Thus, hybrid IDSs leverage network context when evaluating host behavior to improve HMA detection [24].

To improve IDS attack detection, a collaborative decision-making strategy is used through peer information exchange and attack confirmation [25, 26, 27, 28, 29]. However, this raises data privacy concerns between collaborators [30, 31]. Such collaborations follow three strategies, namely, centralized, hierarchical, and distributed. The centralized strategy achieves the best attack detection accuracy but suffers from high network communication overhead and a single point of failure [32, 33]. The hierarchical strategy has a medium attack detection rate and communication overhead, but has several points of failure [34, 35]. The distributed strategy incurs a low communication overhead, has no single point of failure, and is most suitable in Industrial IoT (IIoT) networks, but has a low attack detection rate [36, 37, 38, 39, 40, 31]. To get a high attack detection rate in the distributed design, multihop collaborations [37, 39] and heuristically optimal collaborator selection [31] have been suggested. However, as multihop collaboration strategies increase network communication overhead and heuristic strategies require dense networks to select optimal collaborators, they are unsuitable for resource-constrained IIoT networks. Further, to ensure that attackers do not collude against truthful collaborators, resource-intensive trust management mechanisms are utilized in the aggregation of collaboration responses [37, 38, 39, 40, 31], and often involve certification authorities (CA) in the joining process of new devices, introducing single points of failure [37, 38, 40]. In addition, none of the aforementioned collaboration strategies consider the number of collaborators voting against the suspected device as an input when calculating the penalty to the device's trust value.

Figure 1 shows a sparsely honest network with untruthful collaboration issues in the existing solutions. Here, devices 5, 8, and 9 (interchangeably, nodes or motes) are malicious colluders (represented by underlined node numbers). Say a mimicry attack has happened on node 2, and the HIDS of node 2 cannot give any conclusive decision. So, node 2 participates in a collaboration, suspecting that node 4 is the source of the attack. Now, node 2 requests information on the confirmed and suspected attacker lists from its immediate neighbors, ignoring the DODAG topology, by broadcasting a new control packet. In the first hop neighbors shown in Figure 1 with dashed circles for nodes 2 and 5, there is only one truthful collaborator (node 1) and one untruthful collaborator (node 5). Therefore, collaboration is unsuccessful since a majority cannot be obtained. Similarly, after including the second-hop neighbors in the collaboration process, shown in Figure 1 with double-circled nodes, there are three truthful and three untruthful collaborators, making a majority impossible in this case as well. Thus, in networks with sparsely honest collaborators, a collusion attack nullifies the benefits of collaboration and adversely affects HMA detection accuracy. In addition, there is a significant communication overhead when all responses are directed towards node 2. Furthermore, collaboration requests and responses are not encrypted due to the energy constraints of IoT nodes [41], compromising the privacy of collaborating peers. Moreover, each node continuously overhears its neighbors to build trust, unnecessarily consuming resources. In brief, the existing solutions for HMA detection have one or more challenges in their operations, such as: (i) they can make decisions only with honest collaborators in the majority, (ii) they do not incorporate a hybrid IDS scheme using a fully distributed detection strategy, (iii) they do not address data privacy concerns between

collaborating organizations, (iv) they have high communication overhead, (v) they have low HMA detection accuracy, (vi) they implement resource-intensive trust management mechanisms, and (vii) they ignore the number of devices voting against a suspected node while aggregating collaboration responses.

Therefore, this work proposes a **L**ightweight d**I**stributed collabora**T**ive IDS for **M**imicry-based intr**U**sion detection in **S**parsely honest IIoT networks (LITMUS) to address these issues. LITMUS uses a hybrid IDS scheme for HMA detection. Existing works place both the HIDS and NIDS components on the same device that they monitor, or place the NIDS on the router. However, to monitor a device, this work places one HIDS component on that device and multiple NIDS components on its peer devices. Thus, from an operational perspective, IDS components are shared across devices. Additionally, LITMUS uses a communication scheme for addressing data privacy concerns. Existing works mention the use of bloom filters for communication, but their implementation details are not available. In this work, bloom filters for collaborative communication reduction and lightweight data integrity evaluation are proposed. Furthermore, LITMUS uses a frequency-aware collaboration scheme for response aggregation in sparsely honest networks. Existing schemes employ a binary result-based response aggregation. In this work, a novel frequency-aware response aggregation mechanism is used to acknowledge the number of peers who vote against an attacker device. This enables early detection of stealth attacks. Existing works require several honest collaborators to reach a majority-based decision on the attacker's status. In this work, a majority of honest collaborators is not required for decision-making. This enables intrusion detection in networks with sparse honest collaborators. Moreover, LITMUS uses a lightweight trust management mechanism to protect against insider attackers. Existing works employ resource-intensive neighbor behavior observation and packet sniffing techniques to establish trust. In this work, resource-intensive peer behavior modeling has been replaced with social behavior, historical interaction, and challenge-response-based trusts integrated into a single overall trust value to amplify resource conservation. The contributions of this article are as follows:

- A hybrid lightweight and distributed collaborative IDS design is proposed for detecting HMA attacks. The IDS for detecting a device's maliciousness is distributed across the device and its neighbors.
- A bloom filter-based communication scheme is proposed that preserves data privacy during information collaboration and reduces communication overhead.
- A frequency-aware collaboration scheme is proposed for aggregating responses to collaboration requests, resulting in the faster detection of stealthy HMA attacks.
- The proposed collaboration scheme works in sparsely honest networks, i.e., with only a few honest peers that may not always form a majority during collaboration.
- A lightweight trust scheme is proposed that combines multiple trust types into a single trust value to protect against various attacks in collaborative settings.

The remainder of this article is organized as follows. Section 2 summarizes the existing works in IDS collaboration for HMA detection in IIoT networks. Section 3 presents the proposed solution. Section 4 presents the evaluation results of the proposed solution, and Section 5 presents the conclusion.

2. Related Work

As more devices join IoT networks, the available attack surface expands [2], making novel attacks feasible and necessitating the deployment of IDSs [4, 5, 6]. HMAs are advanced attacks that mimic legitimate network and device behaviors to evade detection while carrying out malicious activities on the host [7, 8].

HMA Detection: To detect malicious activities, IDSs may rely on attack signatures or predefined rules [9]. Although such approaches provide low false alerts, they cannot detect novel attacks. To detect such novel attacks, IDSs rely on anomaly detection, i.e., deviations in device behavioral patterns [17, 11]. Hybrid IDSs combine these detection schemes and can detect both existing and novel attacks [12]. Anomaly-based detection mechanisms produce a high number of false alerts. To improve detection results, anomaly-based detectors often use ML/DL tools [19, 20, 21]. IDS equipped with ML-based online learning mechanisms can help them self-evolve to detect multi-step stealthy HMA attacks [22, 23]. IDSs can monitor network communications (NIDS) [10, 14, 15] or host device behavior (HIDS) [16, 11] to detect suspected attacks. However, HMAs are frequently undetected by NIDSs [13] because they have a low network footprint. In addition, HIDSs [42] have low HMA detection rates because they lack a network footprint as

Table 1
Summary of IDS Collaboration for HMA Detection in Industrial IoT Networks

IDS Scheme	HMA Accuracy	Resource Consumption	Com. Overhead	Com. Privacy	Frequency Awareness	Sparse Honesty	No Cert. Authority	Peer Trust Management
[11]	Low	Low	×	×	×	×	✓	×
[14]	Low	High	×	×	×	×	×	×
[15]	Low	Low	×	×	×	×	×	×
[17]	Medium	High	×	×	×	×	×	×
[18]	Low	Medium	Medium	×	×	×	×	×
[25]	High	Medium	Medium	×	×	×	✓	×
[29]	Low	Medium	Medium	×	×	×	×	C, S
[31]	×	Low	Low	✓	×	×	✓	R
[32]	×	High	High	×	×	✓	✓	×
[33]	×	High	High	×	×	✓	✓	×
[34]	×	Medium	Medium	×	×	✓	×	×
[35]	×	Medium	Medium	×	×	✓	×	×
[36]	×	Medium	Low	✓	×	×	✓	B
[37]	Medium	Medium	Medium	×	×	×	×	C, S
[38]	×	Medium	Medium	×	×	×	×	H, S
[39]	×	Medium	Medium	×	×	×	✓	S
[40]	×	Medium	Medium	×	×	×	×	C, S
This	High	Low	Low	✓	✓	✓	✓	C, H, S

Com. = Communication, Cert. = Certificate, B = Blockchain, C = Challenge, H = History, R = Random, S = Social.

additional context for attack analysis. Hybrid IDSs [17] use available network context while analyzing host behavior for possible attacks [24]. Often, the network and host behavior analyzers in hybrid IDSs are placed in the same host device to increase detection accuracy and reduce communication overhead between them [17].

Collaborative IDS: To increase the accuracy of HMA detection by the IDSs, collaboration has emerged as a successful strategy [25, 27, 28, 29, 30, 31]. IDS agents exchange information to detect potential attack attempts. Three strategies in collaboration, namely, centralized, hierarchical (decentralized), and distributed, are considered. The centralized strategy provides high detection accuracy at the cost of high communication overhead and a single point of failure [32, 33]. The decentralized strategy provides medium detection accuracy, incurs medium communication overhead, and has multiple points of failure [34, 35]. The fully distributed strategy incurs the least communication overhead of all the strategies but compromises on attack detection accuracy [36, 37, 38, 39, 40, 31]. To improve the detection accuracy of the distributed strategy, multihop collaboration has been proposed [37, 39]. However, the multihop collaborators incur high communication overhead. Heuristically optimal collaborator selection has been proposed to maximize the number of collaborators with the required information [31]. However, such algorithms require a dense network to select optimal collaborators from neighbors. In addition, they do not use trust-based aggregation of collaboration responses, allowing untrusted neighbors to participate equally in the process, which raises security concerns.

Trust-Based Collaboration: To address the issue of the existence of malicious collaborators in the network, collaborative IDSs employ trust management mechanisms that rate other IDS agents based on their social behavior [37, 38, 39, 40], historical interactions [37, 38, 40], attack detection competency [37, 40], and heuristically optimal random collaborations [31]. CAs are involved in the joining process of new devices [37, 38, 40] to defend the network against malicious devices that leave and rejoin the network to erase their malicious interaction history. However, the availability of a CA is an essential component for the proper functioning of that scheme and a single point of failure [37, 38]. Social behavior tracing and packet-handling statistics for peers involve overhearing and partial processing of neighbor communications, consuming the tracker's resources [37, 38, 39, 40]. Attack detection competency determination among peers involves a challenge-and-response system that incurs its own communication overhead [37, 38, 40].

Summary: Most of the above works do not consider data privacy while collaborating. A majority of the aforementioned works do not detect HMAs. Solutions that detect HMAs incur high communication and resource

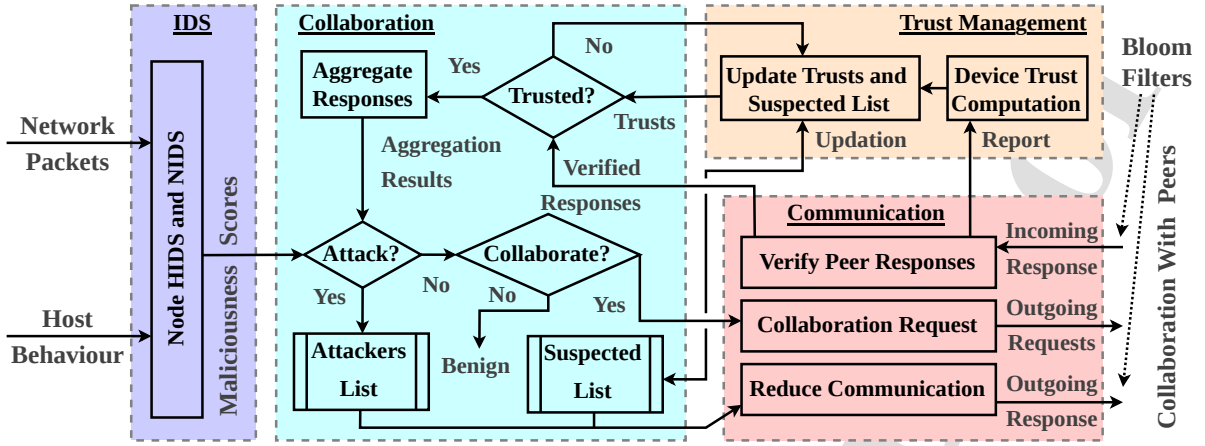


Figure 2: Schematic view of the modules of LITMUS.

overheads and do not implement the scheme as a fully distributed detection strategy. Distributed schemes have low communication overhead but low attack detection accuracy. Therefore, they attempt to achieve better accuracy by collaboration, increasing their communication overhead. Existing solutions utilize resource-intensive trust accumulation and computation mechanisms. They also deploy a weighted-majority voting rule to aggregate the received responses, which ignores the number of collaborators who agree on the attack under consideration. These voting rules do not work well in networks with low honesty, as obtaining a majority is not guaranteed. Finally, the summary of the notable features of the existing works is presented in Table 1.

3. Proposed Methodology

This work proposes a **L**ightweight d**I**stributed collabora**T**ive IDS for **M**imicry-based intr**U**sion detection in **S**parsely honest IIoT networks (LITMUS). At the expiry of each predefined time period t (henceforth, cycle), LITMUS executes to detect attacks on the host device. LITMUS consists of four modules, namely, the *hybrid IDS module* that provides host and network-based intrusion detection capabilities (discussed in Section 3.2), the *communication module* that takes care of the communications, data privacy, and response integrity checks (discussed in Section 3.3), the *collaboration module* that decides on information collaboration and aggregation of responses (discussed in Section 3.4), and the *trust management module* that protects the network against insider attacks using trust scores for peers (discussed in Section 3.5). The Section 3.1 discusses the attack model used in this work.

Figure 2 presents the schematic view of the proposed framework in a device. The host behavior feature vector is the HIDS input, generating an attack score for the host. The incoming data packets are inputs to the NIDS, generating scores for each sender. These scores are fed to the collaboration module, which detects attacks. If the scores are inconclusive of an attack but require further investigation, the communication module requests peer collaboration. This module ensures data privacy when collaborating with other devices. When a collaboration request is received by the communication module of a node, it sends the lists of confirmed and suspected attackers as bloom filters to the requester. When a node receives responses corresponding to its collaboration request, the communication module performs response integrity checks and passes them on to the collaboration module. These integrity check reports are sent to the trust management module, which computes device trust values used for response aggregation and attack detection.

Table 2 summarizes the novelty in the contribution of the proposed mechanism LITMUS, and its delineation from the existing works. The same is presented in line with the various objectives of the four modules of the proposed framework LITMUS. In the intrusion detection (IDS) module, attack detection is performed by the HIDS and the NIDS on the behavior of a single device at a time. LITMUS distributes the task of attack detection of a device onto the HIDS of that device and the multiple NIDSs of its peer devices. Additionally, the existing works use the HIDS and the NIDS of a device only for that device's behavior supervision. LITMUS distributes this over the device (HIDS) and its peers (multiple NIDSs) for collaborative decision-making. In the collaboration module, existing works aggregate peer responses based on trust thresholds. LITMUS uses trust thresholding and data integrity-based response aggregation.

Table 2
Novelty in Contributions and Delineation from Existing Works

Module	Objective	Implementation in Similar Existing Works	Implementation in the Proposed Work
Intrusion Detection	Attack detection	HIDS and NIDS on the device.	NIDS distributed across peers. Collaborative decision-making among the HIDS and peer NIDSs.
	Device supervision	Only self-supervision of devices.	Collaborative peer behavior supervision. Robust peer-based device behavior modeling.
Collaboration	Response aggregation	Trust based thresholding.	Trust and data integrity-based. Immune to several types of attacks and noise.
	Decision-making	Majority voting-based.	Bloom filter-based. Majority not required for decision-making. Works with dishonest peers.
	Attacker detection	Binary: Attack or Benign.	Frequency-aware attack detection. Early detection of stealth attacks.
	Dishonesty resilience	Not implemented.	Continuous peer trust monitoring, response consistency checking, and bloom filter-based aggregation.
Communication	Resource conservation	Bloom filter. Details not available.	Bloom filter implementation. Ensures lightweight data privacy and reduced communication overhead.
	Response Integrity	Not implemented.	Fast and efficient bloom filter-based integrity checks. Immune to attacks and noise.
Trust management	Peer trust establishment	Peer monitoring and packet sniffing.	Trust based on historical interactions, social behavior, and challenge response. Conserves resources.
	Trust value integration	Linear with experimental coefficients.	Linear averaging. Equivalent priority to all trust types. Faster trust depletion for attackers.

This provides immunity against several attacks and noise as discussed in Section 3.5. Additionally, existing works use majority voting-based decision-making, which requires a majority of truthful peers. LITMUS uses bloom filters for attack detection. This works in networks with fewer honest peers and does not require a majority for decision-making. Furthermore, existing works produce binary output for attack detection. LITMUS uses frequency-aware attack detection to address the number of peers voting against an attacker. This results in faster detection of stealth attacks. Moreover, existing works do not address network resilience in the presence of a majority of dishonest collaborators. LITMUS uses continuous peer trust monitoring, response consistency checks, and bloom filter-based collaboration response aggregation to work with dishonest collaborators in the majority.

In the communication module, existing works use bloom filters to reduce communication overhead [31]. However, the implementation details for this scheme are not available. LITMUS uses bloom filters for lightweight communication and data privacy with detailed implementation. Additionally, existing works do not implement response check mechanisms. LITMUS implements response integrity checks using bloom filters. Furthermore, it provides an efficient and lightweight mechanism for response integrity checks. This provides LITMUS immunity against several attacks and noise as discussed in Section 3.5. In the trust management module, existing works use peer monitoring and packet sniffing to establish peer trust. This is very resource-intensive as it increases the radio duty cycle of the monitoring devices manyfold. LITMUS uses trust based on historical interactions, social behavior, and challenge responses. This does not affect the radio duty cycle of the monitoring device and conserves its resources. Additionally, existing works integrate various trust values linearly using experimental coefficients. LITMUS uses linear averaging of trust values using unit coefficients. This ensures equal priority to all types of trusts, resulting in faster trust depletion of attacker devices. These different modules are discussed subsequently. Table 3 presents the frequently used symbols.

3.1. Host Oriented Mimicry Attack Model

HMAs are classified as advanced attacks that mimic legitimate host and network behavior to evade detection mechanisms while carrying out malicious activities on host devices [7, 8]. Several implementations of such attacks

Table 3
Frequently Used Symbols and their Descriptions

Symbol	Description	Symbol	Description
A	List of confirmed attackers of a node	C_{Col}	Minimum score required for collaboration
P	List of suspected attackers of a node	Cnf	Verifications required for attacker confirmation
S	List of collaborators of a node	F_N	Network correlation-based featuremap (clustering)
U	List of immediate neighbors of a node	P_{Fr}	Forgetting factor for suspected attacker node list
$AE_{H/N}$	Autoencoder (AE) of HIDS/NIDS	t	Repeating time period (cycle) in deployment
$\vec{B}^t$	Bloom filter sent at cycle t	T_{Col}	Minimum trust gained on successful collaboration
$C_{H/N}$	HIDS/NIDS score	T_{Min}	Minimum trust to not be a suspected attacker
$\mathcal{H}_i$	HIDS in node i	T_{Res}	Minimum trust for response consideration
$\mathcal{N}_i$	NIDS in node i	$X_{H/N}$	Training phase host or network behavior
Res	Neighbor collaboration responses received	$Y_{H/N}$	Deployment phase host or network behavior
T_{New}	Initial trust for new devices	$Tol_{H/N}$	HIDS/NIDS attack tolerance before raising alarm
T_{Tot}	List of hybrid trust on collaborators	V	Number of host devices in the network

exist in the literature that assume the detection algorithm is available in the open domain, and the engineered attacks can be fed to the algorithm to demonstrate attack evasion. The attack is accordingly modified at each iteration to arrive at a version that successfully evades the detection algorithm. This work designs the attack by executing small pieces of attack code at randomly chosen, sparse times during legitimate host activity [43]. The attack code is therefore already present on the host device and is classified as an insider attack. This attack aims to increase the frequency of communication between the affected devices and their parent devices, thereby draining their energy faster.

3.2. Proposed IDS for HMA Detection

Influenced by the designs of HIDS in [17] and NIDS in [44], the proposed positional hybrid IDS model comprises a single HIDS and multiple distributed NIDSs. As an example, the decision of the maliciousness of node 4 in Figure 1 is taken by any of its neighbors using both the scores of the HIDS resident in node 4 itself (i.e., $\mathcal{H}_4$), and the NIDSs present in the neighbors of node 4 (e.g., $\mathcal{N}_2$, $\mathcal{N}_7$, etc.). On the other hand, to detect mimicry attacks on a host, say node 2, the host node uses its HIDS (i.e., $\mathcal{H}_2$) and the NIDSs present in its neighbors till 2-hops (e.g., $\mathcal{N}_1$, $\mathcal{N}_3$, $\mathcal{N}_7$, $\mathcal{N}_8$, etc.). The HIDS analyzes host behavior to detect attack attempts on the host device. The NIDSs distributed across peer devices accept incoming data packets per cycle t from their immediate neighbors and group them by sender. Thereafter, each NIDS processes these sender-wise groups separately to detect attack attempts. Therefore, the malignancy of each node is checked at two levels. Firstly, the HIDS resident on the host node itself checks for any malicious behavior on the host. Secondly, NIDSs in the neighbors that receive data packets from this host can detect any malicious behavior by this host. These results are aggregated via collaboration between peer devices.

In the network presented in Figure 1, each node runs its own HIDS ($\mathcal{H}$) for host behaviors and its own NIDS ($\mathcal{N}$) for incoming packets from its immediate neighbors. Assume at any cycle t, node 2 runs its NIDS $\mathcal{N}_2$ and gets a score $C_N^4 < 1$ corresponding to node 4. Since this score is inconclusive about an attack, node 2 decides to collaborate with its neighbors to obtain information about the suspected attacker, node 4, if the score exceeds a lower bound for collaboration, i.e., $C_N^4 > C_{Col}$. This information is requested via a control message broadcast that ignores the DODAG topology to reach all the neighbors of node 2 within its two-hop communication range, using the time-to-live (TTL) field of the control packets. This is illustrated in Figure 1 with dashed circles around nodes 2 and 5 that depict their single-hop communication ranges. The proposed collaboration scheme requires a dedicated communication module for the task described above. In Figure 1, node 2 has the single hop neighbors as nodes 1, 4, and 5.

Using the second hop, the neighborhood of node 2 is expanded to include nodes 3, 7, 8, and 9. The decision of whether node 4 is malicious is now taken by node 2 using the inputs from the HIDS of node 4 ($\mathcal{H}_4$) and the NIDSs of node 2 itself ($\mathcal{N}_2$) and its two-hop neighbors ($\mathcal{N}_1$, $\mathcal{N}_3$, $\mathcal{N}_7$, $\mathcal{N}_5$, $\mathcal{N}_8$, and $\mathcal{N}_9$). Definitely, not all nodes in this neighborhood will have information relevant to this decision (e.g., node 9), as not all nodes may receive any data packet traveling from/through node 4. However, the two-hop range allows node 2 to reach the direct neighbors of node 4 who most certainly have information relevant to this decision (e.g., nodes 7 and 8). In addition, the collaboration request packet does not contain information about the suspected node to keep its identity anonymous and stop colluders from voting against it. Therefore, node 9 also responds to the collaboration request as it does not know about the

Algorithm 1: HIDS Training and Deployment.

Input: Tol_H, X_H, Y_H
Output: C_H
1 **create** autoencoder AE_H
2 **train** AE_H using X_H
3 $R_H^{th} \leftarrow$ max reconstruction error $max(AE_H(X_H))$
4 **foreach** *cycle t* **do**
5 $\quad R_H \leftarrow$ recon. error $AE_H(Y_H)$
6 $\quad V_H \leftarrow$ **count** threshold violations $R_H > R_H^{th}$
7 $\quad C_H \leftarrow \min(1, V_H/Tol_H)$

suspected node either. During aggregation of the collaboration responses, such irrelevant information is rejected by node 2 (detailed discussion in Section 3.4). Although the information from the HIDS on node 4 cannot be trusted, it is useful in two ways. Firstly, it can detect HMAs running on node 4 because the host behavior features are only available on the host node. Secondly, it reinforces the malignancy of node 4 if its value indicates an attack. Any misreported values of $\mathcal{H}_4$ are discarded after cross-checking against values from peer NIDSs.

The CPU energy consumption of a node and the energy consumption of packet transmission are the features used by the HIDS. For the NIDS, the consecutive differences in packet timestamp, packet sizes, source and destination IP addresses converted using ordinal encoding [45], and the protocol, source, and destination ports using one-hot encoding [45] are used as input features. The IDSs are preceded by correlation-based feature selectors to reduce the number of input features. Both IDSs are autoencoders (AEs) [46], with the NIDS being an ensemble of AEs, as AEs perform well at unseen pattern detection, and shallow ensembles reduce trainable parameters [47]. The selected host features act as inputs to the HIDS. This HIDS consists of a long short-term memory (LSTM) [48] layer, which performs well with sequential data. For faster data reconstruction, this AE has a fully connected (FC) [49] based bottleneck layer with a dimension of one and another FC output layer that reproduces the inputs [17]. Instead of a binary result (*benign* or *attack*), the IDSs are modified to generate a score of maliciousness, which is the ratio of the number of violations of the reconstruction error threshold (R_H^{th} or R_N^{th}) over the attack tolerance threshold (Tol_H or Tol_N). Algorithm 1 presents the training and deployment of the HIDS run by each node in the network. The inputs to the algorithm are the number of host attack detections beyond which an attack alert is raised (Tol_H), the benign host behavior after feature selection (X_H), and the deploy-time host behavior features (Y_H) for cycle t. The algorithm's output is the cycle's host attack score C_H. An AE is created and trained using the benign behavior, and the maximum benign reconstruction error for the trained AE (R_H^{th}) is used as the threshold for deploy time reconstruction errors (shown in lines 1 to 3). For each deploy cycle t, the number of reconstruction errors above R_H^{th} is counted as V_H to generate the score for cycle t as C_H (shown in lines 5 to 7).

The NIDS is an ensemble of FC AEs. Network packets are grouped by source device; useful features are selected and clustered using correlation to form a feature map. This clustering reduces the NIDS's total computational requirements by mapping each cluster to an ensemble AE [18]. Each AE then regenerates the input feature using a bottleneck layer with its dimension equal to the square root of the input dimension. Algorithm 2 presents the steps involved in training and deploying the NIDS, which is also run on each network node. The algorithm accepts as input the maximum number of network attack detections beyond which an attack alert is raised (Tol_N), the featuremap generated by correlation-based feature clustering (F_N), the benign network packets after feature selection (X_N), and the deploy time network packet features (Y_H) for the cycle t. The algorithm's output is the network attack score C_N^u for that cycle for each immediate neighbor $u \in U \subseteq S$. For each cluster in the featuremap, an ensemble AE is created and trained with the corresponding benign data (shown in lines 3 and 4). The reconstruction errors from the trained ensemble are used to train an aggregator AE, and its maximum reconstruction error is obtained as R_N^{th} (shown in lines 5 to 8). From here, the following steps are repeated for every deployment cycle t. The total ensemble reconstruction error for each immediate neighbor $u \in U$ is fed to the aggregator AE (shown in lines 10 to 15). The number of instances where the aggregator reconstruction error is greater than R_N^{th} is obtained, and the attack score C_N^u for the source u for that cycle t is computed (shown in lines 16 to 19).

Algorithm 2: NIDS Training and Deployment.

Input: Tol_N, F_N, X_N, Y_N
Output: C_N^U

$R_N^{en} \leftarrow \phi$
foreach *feature cluster* $f \in F_N$ **do**
  create autoencoder AE_N^f
  train AE_N^f using X_N^f
  $R_N^{en} \leftarrow R_N^{en} \cup$ { reconstruction error $AE_N^f(X_N^f)$}
create autoencoder AE_N
train AE_N using R_N^{en}
$R_N^{th} \leftarrow$ max reconstruction error $max(AE_N(R_N^{en}))$
foreach *cycle* t **do**
  $U \leftarrow$ get immediate neighbor devices from Y_N
  $C_N^U \leftarrow \phi$
  foreach *neighbor* $u \in U$ **do**
    $R_N^{en} \leftarrow \phi$
    foreach *feature cluster* $f \in F_N$ **do**
      $R_N^{en} \leftarrow R_N^{en} \cup$ { recon. error $AE_N^f(Y_N^f)$}
    $R_N \leftarrow$ max recon. error $max(AE_N(R_N^{en}))$
    $V_N \leftarrow$ **count** threshold violations $R_N > R_N^{th}$
    $C_N^u \leftarrow min(1, V_N/Tol_N)$
    $C_N^U \leftarrow C_N^U \cup \{C_N^u\}$

3.3. The Communication Module

This module communicates with its two-hop neighbors, as discussed in Section 3.2, in a sparsely honest IIoT network environment. It sends the collaboration requests as special control packets that do not contain any attacker information. However, when replying to such a request, a bloom filter [50, 51, 31] is used. Bloom filters are one-way probabilistic data structures that efficiently store data. They never produce false negatives, and false positives can be mitigated by verification with multiple filters [52]. Bloom filters consist of bit vectors with a size determined by the maximum expected number of values to be inserted and the desired false-positive rate. Initially, all bits are set to 0. Data is divided into k parts and hashed using k different hash functions to provide k indices of the bit vector. The values at these indices are set to 1 for insertion and checked against 1 for verification. Bloom filters provide probabilistic verification and do not allow deletions of values. Encoding communications as bloom filters ensures data privacy for the collaborating organizations in IIoT networks. The false positive probability of a bloom filter is:

$$q = \left(1 - (1 - (1/m))^{kn}\right)^k \approx \left(1 - e^{-kn/m}\right)^k, \tag{1}$$

where m is the number of bits in the bloom filter, and n is the number of elements expected to be inserted into the filter. Using an optimal number of hash functions $k = (m/n)\ln 2$, the number of bits required in the bloom filter for a desired n element insertions is given by:

$$m = -(n \ln q)/(\ln 2)^2. \tag{2}$$

When sending a response to a request, the communication module converts the lists of confirmed (A) and suspected (P) attackers to bloom filters [50], $\vec{B}_A^t$, and $\vec{B}_P^t$, respectively, by the aforementioned process. A UNION (lossless) operation is performed between these two filters, removing duplicate entries. The resultant bloom filter, given by $\vec{B}_A^t \cup \vec{B}_P^t$, is communicated as a response to the request message. To receive responses, the communication module records the most recent bloom filters received from each collaborator. Say, for the previously received bloom filter $\vec{B}^t$ of a collaborator,

Algorithm 3: Aggregation of Responses.

Input: $A, P, Res, T_{Tot}, Cnf, Tol_N, T_{Res}$
Output: A, P

```
1  Q^P ← 0⃗
2  S ← sources in response list Res
3  foreach suspected p ∈ P do
4      foreach source s ∈ S do
5          if p ∈ Res^s and T^s_Tot > T_Res then
6              Q^p ← Q^p + 1
7      if Q^p ≥ Cnf then P ← P ∪ {{p}_×Q^p}
8  foreach suspected p ∈ set(P) do
9      if |P^p| > Tol_N then
10         A ← A ∪ {p}
11         P ← P − {{p}_×Q^p}
```

x different bits are set to 1. Now, since bloom filters do not support deletions, at least these x bits must be set to 1 in the device's upcoming responses $\vec{B}^{t+\epsilon}$ for $\epsilon \geq 0$, i.e., $\vec{B}^t \subseteq \vec{B}^{t+1}$. Therefore, a bitwise XOR of the current and previous responses results in the x bits being 0. Performing a bitwise AND operation on this result with the previously stored $\vec{B}^t$, which has these x bits set to 1 and the remaining $(m - x)$ bits set to 0, will result in a $\vec{0}$ indicating data integrity. If the response message fails this test, that report is sent to the trust management module as discussed in Section 3.5. Passed responses are aggregated in the collaboration module.

$$(\vec{B}^{t+1} \oplus \vec{B}^t) \otimes \vec{B}^t \stackrel{?}{=} \vec{0}. \tag{3}$$

3.4. The Collaboration Module

The collaboration module handles three tasks, namely, sending collaboration requests, sending responses to collaboration requests, and aggregating received responses. This module receives the attack scores from the IDS module and decides to collaborate with other IDSs if the scores are in the open range $(C_{Col}, 1)$. An *empty control packet* containing no attacker information is sent to the neighbors within a two-hop vicinity, requesting their attack-detection analysis. Each receiver device reverts to this message, which includes a bloom filter containing its suspected and confirmed malicious device lists. The communication module verifies the integrity of neighbor responses and forwards them to the collaboration module for trust-based aggregation. Since bloom filter-based verification is probabilistic, the false positive rate is reduced by verifying attacker data across multiple responses [52], as discussed in Section 4.2.3. Existing solutions require a majority of honest peers, i.e., in the interval $(|S|/2, |S|]$, for decision-making. In contrast, the number of bloom filter checks Cnf required to verify an attacker need not be a majority ($0 < Cnf \leq |S|$), working well for sparsely honest IIoT networks. When aggregating responses with trust values above the required trust threshold T_{Res}, the number of positive matches is taken as the number of times the device was detected as a suspected attacker. Thus, the outcome of the aggregation process increases the number of attack attempts by the number of response bloom filter matches, rather than incrementing by 1 (as in a binary majority voting algorithm). Therefore, the aggregation process is frequency-aware of attack attempts against the host device's collaborators.

Algorithm 3 presents the response aggregation process. The attackers' list (A), the suspected attackers' list (P), the received responses (Res), the collaborator trusts (T_{Tot}), the number of bloom filters to verify for the desired maximum false positive rate (Cnf), the number of attack attempts detected after which an attack alert is raised (Tol_N), and the collaborator trust threshold beyond which a response is aggregated (T_{Res}) are the inputs to the algorithm. Its outputs are the modified confirmed and suspected attackers' lists (A and P). At first, the sources of responses are collected in S, and each suspected attacker is searched in the response of a collaborator with a trust score above the threshold Tol_N (shown in lines 1 to 6). The number of times Q^p of each suspected attacker $p \in P$ is found in such eligible responses must surpass Cnf to ensure a desirable false positive rate. If this threshold is cleared, this suspected attacker is considered a confirmed attacker rather than a false alarm, and the attacker's entry is added Q^p times to the list P (shown in line 7). Thus, the number of attack attempts by attackers in the collaboration community is accounted

for, enabling faster attack detection. Finally, for each suspected attacker, p, its frequency in P is checked against the threshold Tol_N. If the frequency exceeds the threshold, the attacker is moved to the confirmed attackers' list and removed from the list of suspected nodes (shown in lines 10 and 11).

3.5. The Trust Management Module

The proposed trust management module computes and combines three types of trust, namely, challenge response-based trust, social behavior-based trust, and historical interaction-based trust, into one hybrid trust value T_{Tot}. This value is used by the collaboration module for response aggregation.

3.5.1. Historical Interaction Based Trust

The history of personal interactions with other devices is the primary basis of this type of trust. For each collaboration response that passes the response integrity test in the communication module (as shown in Figure 2), the collaborator gains trust in cycle t as:

$$\Delta T^t_{HI} = \max\left(T_{Col}, T^t_{Gain} \times \frac{\exp\left(T^t_{Tot} - 1\right)}{t}\right), \tag{4}$$

where $T^t_{Gain} = T^t_{HI} - (T_{New} - T_{Min})$ is the trust gained post joining the network, T_{New} is the trust of a newly joined device, T_{Min} is the minimum acceptable benign trust value, and T_{Col} is the minimum trust that is awarded on successful collaboration. However, a collaborator can also be penalized under two conditions. Firstly, a collaborator is penalized if its trust value is below a threshold T_{Min}, indicating poor interaction over time. The penalty is adding the collaborator to the list of suspected attackers.

$$P = P \cup \{s\}, \quad \text{where} \quad s \in S. \tag{5}$$

If the total number of entries $|P^s|$ of a collaborator s in the list P is above a threshold Tol_N. This collaborator is declared as a confirmed attacker, and its trust value is given by:

$$T^{t+1}_{HI} = T^t_{HI} \times \left(\frac{Tol_N}{|P^s|}\right)^{\ln\sqrt{t}}. \tag{6}$$

This trust reduction is proportional to the number of cycles elapsed, i.e., devices that misbehave later are penalized more. Trust building is a slower process for new devices than for those with greater experience. However, for the existing solutions, newly joined devices build trust rapidly.

3.5.2. Challenge Response Based Trust

In the proposed mechanism, the collaboration request and its responses serve as challenge and response messages, respectively, with no extra overhead for this type of trust. Since bloom filters are used in the proposed mechanism, either the suspected attacker $p \in P$ is in a filter, or it is not. A device stores each collaborator's last received response as the ideal response and compares the next response against it. These responses are bloom filters that provide a probabilistic response to the requesting device, with a false-positive rate given by Equation (1). Therefore, a subset checking ($\vec{B}^t \subseteq \vec{B}^{t+1}$) on the current ($\vec{B}^{t+1}$) and the previous ($\vec{B}^t$) bloom filters provide an efficient approach for the verification of existence of each suspected attacker in the response by mapping each binary response to multiple indices in the bloom filters and using a single operation on the filters. This is done as described by Equation (3) in Section 3.3 by performing a bitwise XOR of the filters and a bitwise AND of the result with the stored filter. The final result is compared with $\vec{0}$. Using $H(\cdot)$ as the Hamming distance between vectors, the trust value of a collaborator failing this test is:

$$T^{t+1}_{CR} = T^t_{CR} \times \left[1 - \left(\frac{H(\vec{B}^{t+1}, \vec{B}^t)}{x}\right)^{\ln\sqrt{t}}\right]. \tag{7}$$

3.5.3. Social Behavior Based Trust

In contrast to existing schemes, no separate communication or overhearing of neighboring devices is required for computing this trust value, thereby saving device resources and communication overhead. Instead, the relevant information is extracted from the collaboration responses. The frequency-aware nature of response aggregation, as discussed in Section 3.4, addresses this aspect of trust. The proposed IDS module spans multiple devices at two levels: the HIDS monitors the host behavior, and the collaborators' NIDSs monitor the network behavior of the aforementioned host, as discussed in Section 3.2. The lists of suspected and confirmed attackers (P and A) for each collaborator are obtained by the collaboration module as collaboration responses. If voted against by more than Cnf other collaborators, the trust in a suspected attacker $p \in P$ is reduced by multiple added entries of the device p into the list of suspected attackers P.

$$P^p = P^p \cup \{\{p\}_{\times|Res^p|}\}, \tag{8}$$

where $|Res^p|$ is the number of votes against p in the response list Res. This frequency-awareness of the trust mechanism increases the number of attacks detected for the suspected attacker, thereby accelerating attacker detection. The loss of social behavior-based trust for the suspected device is given by:

$$T_{SB}^{t+1} = \frac{T_{SB}^{t}}{\sqrt{1 + |Res^p|}}. \tag{9}$$

3.5.4. Hybrid Overall Trust

Existing solutions use a linear combination of experimental coefficients for different trust scores to compute a device's final trust value. LITMUS combines all trust scores using a linear averaging scheme with unit coefficients for different trust scores. This ensures equivalent priority to all types of trust scores. Additionally, this approach ensures that all three trust-building schemes are equally effective at standalone detection of stealthy attacks. This combination of trust scores into the overall trust score T_{Tot} can be represented using Equations (6), (7) and (9) as:

$$T_{Tot} = \frac{1 \cdot T_{CR} + 1 \cdot T_{HI} + 1 \cdot T_{SB}}{3}. \tag{10}$$

Every trust value is initiated with $T_{New} = 0.5$, so new devices cannot participate in decision-making until their trust values exceed T_{Res}. The trust management module also employs a forgetting mechanism to improve the behavior of suspected attackers. This mechanism removes P_{Fr}% device entries from the start of P. Using $\rho(\cdot)$ for the position of an entry in the list P, this mechanism is given by:

$$P = \left\{ p \mid p \in P; \left\lfloor \frac{\rho(p)}{|P|} \right\rfloor > \frac{P_{Fr}}{100} \times |P| \right\}. \tag{11}$$

4. Performance Analysis and Discussion

In this section, the attack resiliency of the proposed mechanism LITMUS has been discussed. Additionally, the theoretical and experimental analyses of the proposed mechanism LITMUS have been performed.

4.1. Attack Resiliency using Trusts

The trust management module helps collaboration and provides resiliency against various attacks.

4.1.1. Betrayal Attacks

These are attacks in which a node collaborates truthfully until its trust is acceptable to the response aggregation process. Thereafter, it collaborates maliciously. As formulated in Equation (6), in terms of historical interaction-based trust, device misbehavior is increasingly penalized over time. Thus, gaining trust is a slow process, whereas losing trust due to unsuccessful collaborations is fast, deterring betrayal attacks.

Figure 3 shows the attack model for a typical betrayal attack using an attack tree. It can be seen that LITMUS deters the attack using three strategies. Firstly, trust in new motes is initialized to a low value. This prevents new motes from contributing to the response aggregation process. Secondly, the trust-building process is slow. This results in a considerable delay in attack initiation. Thirdly, the trust penalty is aggressive. This rapidly reduces trust in malicious devices, mitigating the damage to the network.

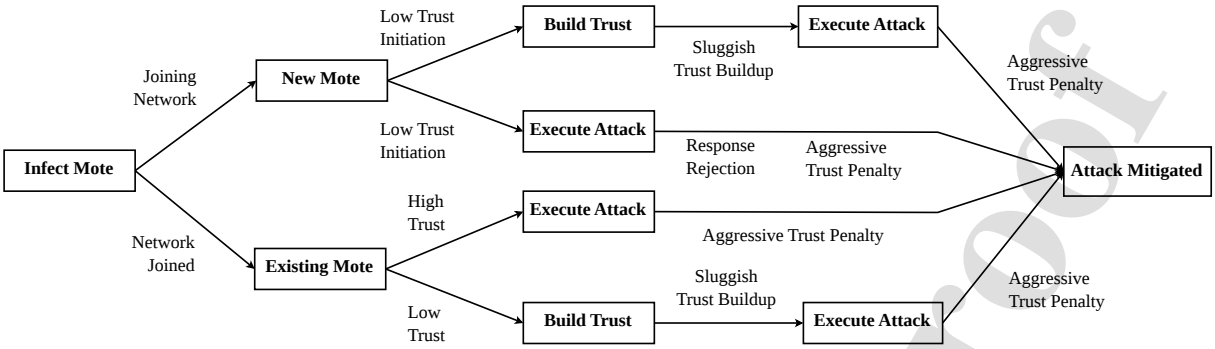


Figure 3: Attack tree for betrayal attacks.

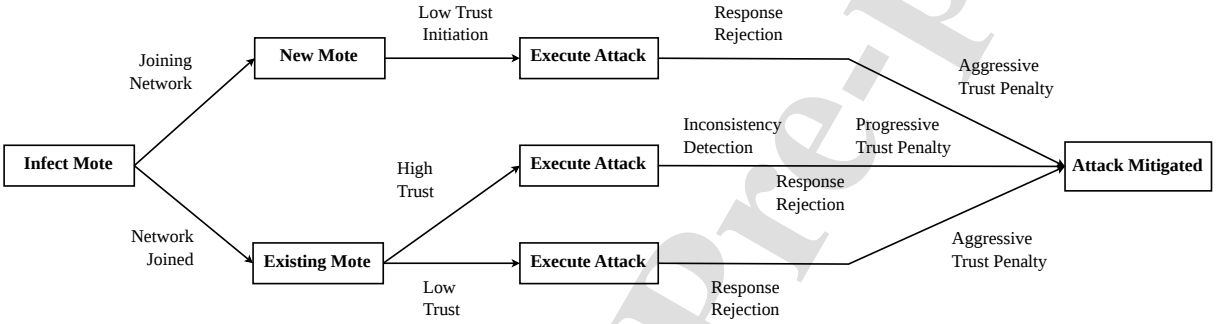


Figure 4: Attack tree for inconsistency attacks.

4.1.2. Inconsistency Attacks

These are attacks in which a collaborator randomly switches between truthful and malicious responses to other collaborators. Existing solutions enable new nodes to build trust rapidly and recover quickly from trust loss resulting from inconsistent collaborations. The hybrid overall trust values in the proposed solution are initiated at 0.5 for newcomer nodes and increment slowly. Inconsistency attracts behavioral trust penalties, making the trust-building process slower, thereby discouraging inconsistency attacks.

Figure 4 shows the attack model for a typical inconsistency attack using an attack tree. It can be seen that LITMUS deters the attack using three strategies. Firstly, trust in new motes is initialized to a low value. This prevents new motes from contributing to the response aggregation process. Secondly, the data integrity check detects inconsistencies in the collaborators' responses. This results in the rejection of these responses during aggregation. Thirdly, the trust penalty is aggressive. This rapidly reduces trust in malicious devices, mitigating the damage to the network.

4.1.3. Collusion Attacks

These are attacks in which multiple malicious nodes coordinate to produce untruthful collaboration responses, each of which downvotes an honest node. Collusion attacks cannot be executed in the proposed framework despite the absence of a trusted CA, because the collaboration requests contain no information about the suspected attacker, and the aggregation process only assembles response information related to that attacker. Therefore, the colluders do not know any victims to downvote, providing resistance up to $(|S| - Cnf)$ colluders.

Figure 5 shows the attack model for a typical collusion attack using an attack tree. It can be seen that LITMUS deters the attack using four strategies. Firstly, trust in new motes is initialized to a low value. This prevents new motes from contributing to the response aggregation process. Secondly, the trust-building process is slow. This results in a considerable delay in attack initiation. Thirdly, the data integrity check detects inconsistencies in the collaborators' responses. This results in the rejection of these responses during aggregation. Fourthly, the trust penalty is aggressive. This rapidly reduces trust in malicious devices, mitigating the damage to the network.

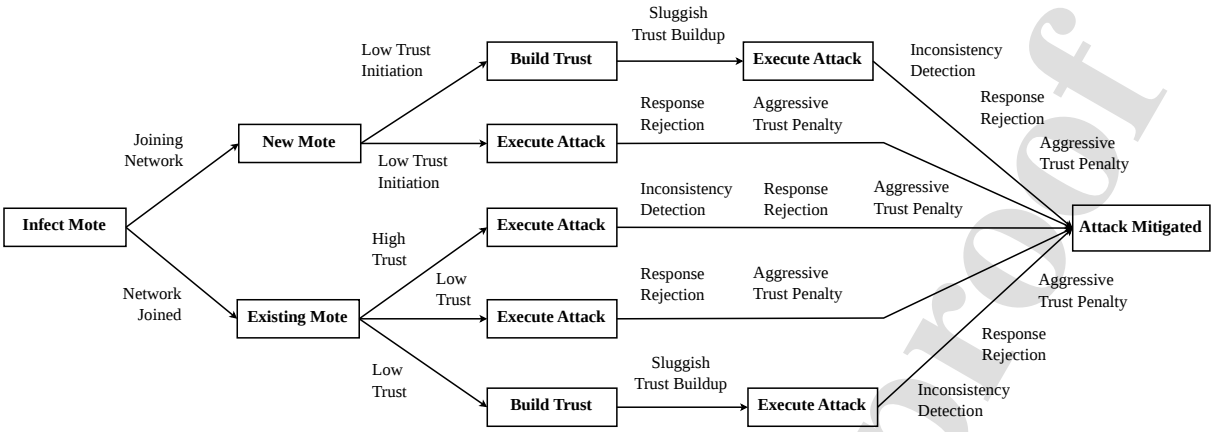


Figure 5: Attack tree for collusion attacks.

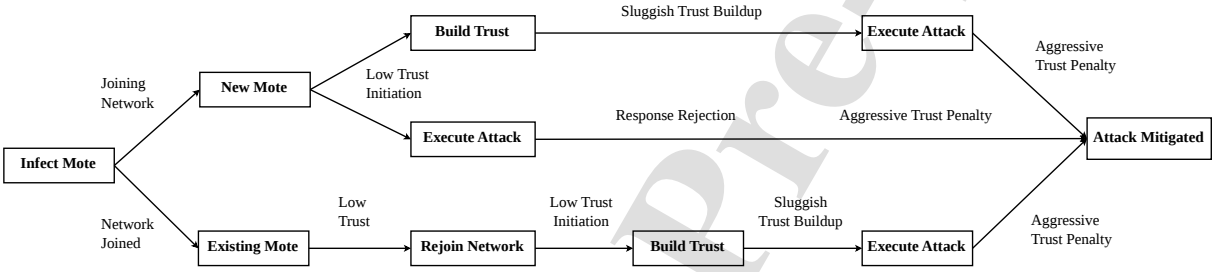


Figure 6: Attack tree for newcomer attacks.

4.1.4. Newcomer (Re-entry) Attacks

These are attacks where a malicious node, detected by the IDS, leaves and rejoins the network as a new node to clear its malicious history and rebuild its trust score. Since hybrid trust values are initialized to 0.5 and only devices with hybrid trust values above T_{Res} can affect the response aggregation detection process, newcomer attacks are not possible.

Figure 6 shows the attack model for a typical newcomer attack using an attack tree. It can be seen that LITMUS deters the attack using three strategies. Firstly, trust in new motes is initialized to a low value. This prevents new motes from contributing to the response aggregation process. Secondly, the trust-building process is slow. This results in a considerable delay in attack initiation. Thirdly, the trust penalty is aggressive. This rapidly reduces trust in malicious devices, mitigating the damage to the network.

4.1.5. Sybil Attacks

These are attacks in which a malicious entity creates multiple false identities (nodes) to evade detection and skew the collaborative response aggregation process. Their primary target, the response aggregation process, works without majority-based voting schemes and is invariant to the number of collaborators beyond the minimum T_{Res} required for attack confirmation. The hybrid overall trust values for new identities (nodes) are initialized at 0.5. In addition, the suspected attacker's information is unavailable for downvoting. Furthermore, the response aggregation process only considers information related to the suspected attackers. Thus, the proposed mechanism is resistant to Sybil attacks.

Figure 7 shows the attack model for a typical Sybil attack using an attack tree. It can be seen that LITMUS deters the attack using four strategies. Firstly, trust in new motes is initialized to a low value. This prevents new motes from contributing to the response aggregation process. Secondly, the trust-building process is slow. This results in a considerable delay in attack initiation. Thirdly, the data integrity check detects inconsistencies in the collaborators' responses. This results in the rejection of these responses during aggregation. Fourthly, the trust penalty is aggressive. This rapidly reduces trust in malicious devices, mitigating the damage to the network.

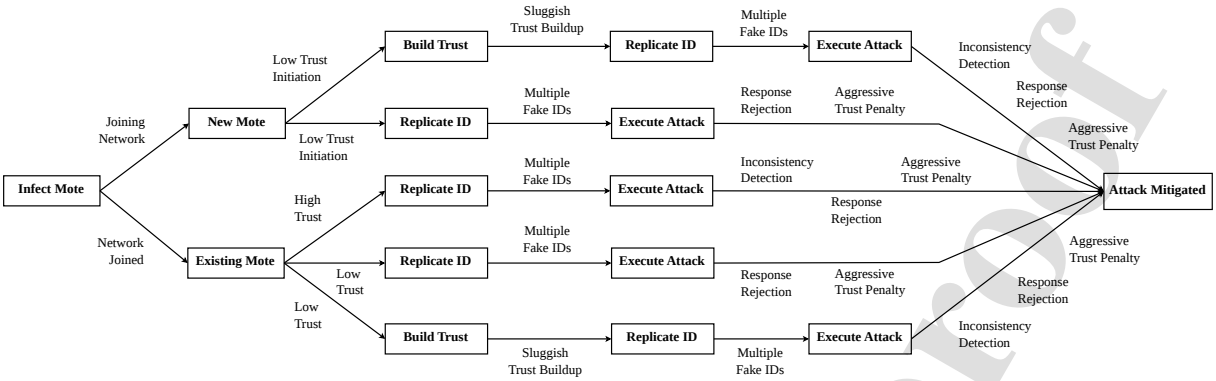


Figure 7: Attack tree for Sybil attacks.

Table 4
Comparison of the Theoretical Analysis Results for the Frameworks

Parameter	CONTRAST	EnergyCIDN	DCIDS–FL	CIPHER	OPTIMIST	LITMUS
Time complexity	$O(V^2)$	$O(V^2)$	$O(V^2)$	$O(V^2)$	$O(V^2)$	$O(V^2)$
Collusion tolerance	$\lfloor \lvert S\rvert/2 \rfloor$	$\lfloor \lvert S\rvert/2 \rfloor$	$\lfloor \lvert S\rvert/2 \rfloor$	$\lfloor \lvert S\rvert/2 \rfloor$	$\lfloor \lvert S\rvert/2 \rfloor$	$\lvert S\rvert - Cnf$
False positive rate	$1-(1-q)^{1+\lfloor \frac{\lvert S\rvert}{2} \rfloor}$	$1-(1-q)^{1+\lfloor \frac{\lvert S\rvert}{2} \rfloor}$	$1-(1-q)^{1+\lfloor \frac{\lvert S\rvert}{2} \rfloor}$	$1-(1-q)^{1+\lfloor \frac{\lvert S\rvert}{2} \rfloor}$	$1-(1-q)^{1+\lfloor \frac{\lvert S\rvert}{2} \rfloor}$	$1-(1-q)^{Cnf}$
Comm. overhead	$O(V^3)$	$O(V^3)$	$O(V^3)$	$O(V^3)$	$O(V^3)$	$O(V^2)$
Energy overhead	$E_T^{CT} > E_T^{Pr}$	$E_T^{CI} \approx E_T^{Pr}$	$E_T^{DC} \leq E_T^{Pr}$	$E_T^{CP} > E_T^{Pr}$	$E_T^{OP} > E_T^{Pr}$	E_T^{Pr}

4.2. Theoretical Performance Analysis

For the theoretical performance analysis, five state-of-the-art mechanisms were chosen, namely, CONTRAST [11], OPTIMIST [14], DCIDS–FL [15], CIPHER [25], and EnergyCIDN [29], and compared with the proposed framework LITMUS. The time complexity, collusion tolerance, false-positive rate, communication overhead, and energy consumption of the proposed mechanism and the state-of-the-art mechanisms have been analyzed here. The summary of the theoretical analysis of these frameworks is presented in Table 4.

4.2.1. Time Complexity

The time complexity of LITMUS is the maximum of the IDS, communication, collaboration, and trust management modules. Therefore, the time complexity of each module is analyzed as follows.

In the IDS module, the HIDS must process a finite number of records at the end of each time period. However, as the host behavior is captured periodically, the number of records is fixed and does not change with the number of devices in the network. Therefore, the HIDS has a time complexity of $O(1)$. On the other hand, the NIDS evaluating a device is distributed over its $|U|$ neighboring devices. Each of these NIDSs has to process a different number of records. However, network behavior is characterized by network traffic that grows with network size and neighbor density. Therefore, the time complexity of each of these NIDSs is $O(|U|)$. Since these NIDSs operate in parallel in different devices, their total time complexity remains $O(|U|)$. Thus, the IDS module has a total time complexity of $O(|U|) \leq O(V)$, as the number of neighbors is always less than the total number of devices in the network ($|U| < V$).

The communication module must prepare a bloom filter for the list of probable attackers P and communicate it to its collaborators U. Additionally, both $|P|$ and $|U|$ grow with increasing network size and neighbor density. Thus, each bloom filter preparation requires $O(|P|)$ value insertions and has to be sent to $|S|$ collaborators. Therefore, the time complexity of the communication module is $O(|P| \cdot |S|)$. However, the number of probable attackers $|P|$ and the number of collaborators of a device $|S|$ are always less than the number of devices in a network V, giving $|P| < V$ and $|S| < V$. Therefore, the time complexity of the communication module becomes $O(|P| \cdot |S|) \leq O(V^2)$.

The collaboration module consults with its peers for all the probable attackers in the list P at a time using bloom filters. Additionally, it aggregates the response data for $|P|$ probable attackers from $|S|$ collaborators. Therefore, it takes $O(|P| \cdot |S|)$ time for response aggregation. However, both the number of probable attackers $|P|$ and the number

of collaborators of a node $|S|$ are less than the number of devices in the network V, giving $|P| < V$ and $|S| < V$. Therefore, the time complexity of the collaboration module becomes $O(|P| \cdot |S|) \leq O(V^2)$.

The trust management module evaluates all $|U|$ neighbor nodes to build trust in them. The number of nodes grows with increasing network size and neighbor density. Thus, the trust management module has a time complexity of $O(|U|)$. Since the number of neighbors $|U|$ is always less than the total number of devices in the network V, giving $|U| < V$. Therefore, the time complexity of the trust management module becomes $O(|U|) \leq O(V)$.

`LITMUS` has an overall time complexity given by the maximum of the time complexities of its four modules.

$$TC = \max\left[O(V), O(V^2), O(V^2), O(V)\right] = O(V^2). \tag{12}$$

As all the state-of-the-art mechanisms except DCIDS–FL operate in the same lines, they have the same time complexity. However, DCIDS–FL does not use a trust mechanism during collaborator response aggregation. Therefore, it does not have the time complexity term for trust establishment. Thus, the overall time complexity of DCIDS–FL is given by:

$$TC = \max\left[O(V), O(V^2), O(V^2)\right] = O(V^2). \tag{13}$$

This is the same as the other state-of-the-art frameworks and that of the proposed mechanism as well.

4.2.2. Collusion Tolerance

All state-of-the-art frameworks use a majority-based voting mechanism to aggregate responses from collaborators. Majority-based voting mechanisms select the majority input as the output. Therefore, such mechanisms rely on the honesty of the majority of its collaborators. Thus, all the state-of-the-art mechanisms provide accurate collaboration results for a range of honest responses out of the total $|S|$ collaboration responses given by:

$$\left|Res_{Honest}^{SOTA}\right| \in \left[\lceil |S|/2 \rceil, |S|\right]. \tag{14}$$

This gives the collusion tolerance of all the state-of-the-art mechanisms as $\lfloor |S|/2 \rfloor$. However, `LITMUS` uses bloom-filter-based collaborative response aggregation rather than the majority-based approach used by the state-of-the-art mechanisms. Bloom filter-based response aggregation does not require a majority of the collaboration responses to be truthful to provide accurate results. The number of bloom filter verifications required to make a decision on the maliciousness of an attacker device depends on the network's desired attack-detection accuracy. Therefore, the range of honest responses required for `LITMUS` is given by:

$$\left|Res_{Honest}^{LITMUS}\right| = Cnf \in \left[1, |S|\right]. \tag{15}$$

Therefore, from 1 to $|S|$ bloom filter confirmations are required to verify the maliciousness of an attacker, depending on the network's accuracy requirement. Thus, the collusion tolerance of `LITMUS` is $|S| - Cnf$.

4.2.3. False Positive Rate

Assume the false positive rate (FPR) of a bloom filter is q, as given in Equation (1). To ensure a lower FPR, each suspected attacker is verified against Cnf bloom filters, making the FPR:

$$q^* = 1 - \left(1 - q\right)^{Cnf}. \tag{16}$$

where $0 < Cnf \leq |S|$. This gives the proposed mechanism resistance against up to $(|S| - Cnf)$ colluding attackers, as opposed to $\lfloor |S|/2 \rfloor$ for OPTIMIST. Assume that, for OPTIMIST, the FPR of individual IDSs is q. Therefore, the FPR for collaboration using majority-based schemes is:

$$q^* = 1 - \left(1 - q\right)^{1+\left\lfloor \frac{|S|}{2} \right\rfloor}. \tag{17}$$

As the other state-of-the-art mechanisms also deploy the same majority-voting-based decision-making approach, their FPR is the same as that of OPTIMIST and is given by Equation (17).

4.2.4. Communication Overhead

The total number of bits communicated, CB_{Tot}, is the sum of the request and response bits. For OPTIMIST with V devices in the network, $|S|$ ($< V$) collaborators, $|W|$ ($< V$) possible attackers, $|P|$ ($< V$) suspected attackers, and $|A|$ ($< V$) confirmed attackers:

$$\begin{aligned} CB_{Tot} &= \Big[|S| \cdot \big\{|W| \cdot |P|\big\}\Big]_2 + \Big[|S| \cdot \big\{|W| \cdot (|P| + |A|)\big\}\Big]_2, \\ &\leq 2 \cdot \Big[|S|^2 \cdot (|P| + |A|)\Big]_2, \\ &\approx O\big(|S|^3\big) \leq O\big(V^3\big), \end{aligned} \tag{18}$$

where $[d]_2$ is the binary representation of a decimal number d and $|W| \leq |S|$. As the other state-of-the-art mechanisms also deploy the same collaboration approach, their total number of bits communicated is also the same as that of OPTIMIST, and is given by Equation (18). For LITMUS, using Equation (2):

$$\begin{aligned} CB_{Tot} &= \Big[|S| \cdot 0\Big]_2 + |S| \cdot \left(\frac{-|S| \ln q}{(\ln 2)^2}\right), \\ &\approx O\big(|S|^2\big) \leq O\big(V^2\big), \end{aligned} \tag{19}$$

which is a magnitude lower than that of the state-of-the-art mechanisms.

4.2.5. Energy Overhead

The total energy overhead (E_T) is the sum of the IDSs (E_I), collaboration (E_{CL}), communication (E_{CO}), response aggregation (E_A), and trust management (E_{TM}) energies. Thus, $E_T = E_I + E_{CL} + E_{CO} + E_A + E_{TM}$. Now, $E_I = E_H + E_N$ for the HIDS and NIDS, respectively. Similarly, $E_{CO} = E_{Tx} + E_{Rx}$ for transmission and reception, respectively. For the proposed mechanism, the energy overhead (E_T^{Pr}) is:

$$E_T^{Pr} = \big(E_H^{Pr} + E_N^{Pr}\big) + E_{CL}^{Pr} + \big(E_{Tx}^{Pr} + E_{Rx}^{Pr}\big) + E_A^{Pr} + E_{TM}^{Pr}. \tag{20}$$

Now, OPTIMIST uses an NIDS as its detector, giving $E_H = 0$. Therefore, its energy consumption (E_T^{OP}) becomes:

$$\begin{aligned} & E_T^{OP} = \big(E_H^{OP} + E_N^{OP}\big) + E_{CL}^{OP} + \big(E_{Tx}^{OP} + E_{Rx}^{OP}\big) + E_A^{OP} + E_{TM}^{OP}, \\ \implies & E_T^{OP} = \big(E_N^{OP}\big) + E_{CL}^{OP} + \big(E_{Tx}^{OP} + E_{Rx}^{OP}\big) + E_A^{OP} + E_{TM}^{OP}. \end{aligned} \tag{21}$$

For the NIDS, $E_N^{Pr} = E_N^{OP}$. A well-designed LSTM-based HIDS with an order of fewer features than an NIDS consumes very low energy [53], giving $E_H^{Pr} \ll E_N^{Pr}$. Thus, $E_I^{Pr} \approx E_N^{Pr} = E_I^{OP}$. Collaboration energy ($E_{CL}$) is the sum of bloom creation and challenge-response verification. Bloom filter-based response checking is much more energy-efficient than suspected-attacker-wise response verification, and bloom filter creation results can be cached, yielding $E_{CL}^{Pr} < E_{CL}^{OP}$. The communication overhead of LITMUS is a magnitude lower than OPTIMIST, giving $E_{Tx}^{Pr} < E_{Tx}^{OP}$ and $E_{Rx}^{Pr} < E_{Rx}^{OP}$. The frequency responsiveness of the response aggregation process adds negligible energy overhead, giving $E_A^{Pr} \approx E_A^{OP}$. Since LITMUS does not overhear neighbor communications for trust building, this gives $E_{TM}^{Pr} < E_{TM}^{OP}$. Therefore, it follows that $E_T^{OP} > E_T^{Pr}$.

Both CONTRAST and CIPHER use the same approach as OPTIMIST and have the same energy overhead as well, giving $E_T^{CT} > E_T^{Pr}$ and $E_T^{CP} > E_T^{Pr}$, respectively. However, DCIDS–FL does not use the trust management module, giving $E_{TM}^{DC} = 0$. Thus, the energy consumption of DCIDS–FL becomes:

$$\begin{aligned} & E_T^{DC} = \big(E_N^{DC}\big) + E_{CL}^{DC} + \big(E_{Tx}^{DC} + E_{Rx}^{DC}\big) + E_A^{DC} + E_{TM}^{DC}, \\ \implies & E_T^{DC} = \big(E_N^{DC}\big) + E_{CL}^{DC} + \big(E_{Tx}^{DC} + E_{Rx}^{DC}\big) + E_A^{DC}, \\ \implies & E_T^{DC} \leq E_T^{Pr}. \end{aligned} \tag{22}$$

Additionally, EnergyCIDN does not continuously monitor neighbor behavior. Therefore, its energy consumption is:

$$\begin{aligned} & E_T^{CI} = \big(E_N^{CI}\big) + E_{CL}^{CI} + \big(E_{Tx}^{CI} + E_{Rx}^{CI}\big) + E_A^{CI} + E_{TM}^{CI}, \\ \implies & E_T^{CI} \approx E_T^{Pr}. \end{aligned} \tag{23}$$

Table 5
Experiment Parameters for Simulation [54] and Testbed [55]

Parameter	Value	Parameter	Value
Radio	UDGM: Distance loss	Topology	Random / Grid
Routing	RPL / RPL lite	Tx range	50 m radius
Platform	Z1 / M3 / A8 / RPi3	Area covered	200×200 m^2
Total motes	10/20/30/40/50/60	Attack motes	10% of total
Total time	10 hours	Attack time	Last 8 hours

4.3. Experimental Performance Analysis

The dataset collection, system implementation, state-of-the-art frameworks, and the evaluation metrics are discussed here. The proposed framework LITMUS has been compared against several state-of-the-art frameworks to evaluate its performance. An ablation study has also been included to evaluate the contributions of its subschemes.

4.3.1. Dataset and Testbed

The Cooja simulator in Contiki-NG [54] and the FIT IoT-LAB testbed [55] were used for data collection with parameters presented in Table 5. An example in Contiki-NG was executed in which devices aperiodically send packets to the root via a multihop topology. Host behavior was captured using the IoT-LAB monitoring profiles and saved as text files. Network behavior was captured as packet capture files and analyzed using Wireshark [56].

4.3.2. System Implementation

The IDSs were implemented in Python and trained on an Intel Xeon Gold 6230 CPU running at 2.10 GHz, with 80 logical cores and 512 GB of memory, for 150 epochs with a batch size of 64, a learning rate of 0.001, and timesteps of 16. Cycles of duration $t = 200$ seconds were used. The trust value for new nodes was $T_{New} = 0.5$, and the minimum acceptable benign trust value was $T_{Min} = 0.1$. The minimum collaborative trust increment was $T_{Col} = 0.01$. The tolerance thresholds for attack attempts for both IDSs were $Tol_{N/H} = 2$. The trust threshold for response aggregation was $T_{Res} = 0.8$. The number of bloom filter validations for attack confirmation was $Cnf = 2$, and the collaboration threshold was $C_{Col} = 0.3$. The Tx and Rx energy consumptions of Z1 motes were used as 17.4 and 18.8 mA, respectively. Power consumption of the different motes was captured using the IoT-LAB monitoring profiles. Dishonesty was introduced into the network so that devices could not obtain a majority of honest collaborators among their neighbors. Additionally, noise was also implemented in the network communications.

4.3.3. State-Of-The-Art Frameworks

Five frameworks were compared, namely, CONTRAST [11], OPTIMIST [14], DCIDS–FL [15], CIPHER [25], and EnergyCIDN [29], with the proposed framework LITMUS. Collaboration was implemented for all the frameworks. All frameworks, except DCIDS–FL, used a trust mechanism for response aggregation.

4.3.4. Evaluation Metrics

The frameworks were evaluated using the number of HMA detections, the accuracy of the detection model, the false positive rate (FPR), the number of bits communicated for collaboration, the power consumed, the number of trusted collaborators per cycle who contribute to decision making, the energy overhead, the memory overhead, the model resident size in memory, and the time consumed for decision making per cycle. The metrics were presented for network size variation and neighborhood density. Additionally, a sensitivity analysis of the proposed framework for different values of Bloom filter parameters was also presented. Furthermore, an ablation study was included to evaluate the contributions of the framework's components.

4.3.5. Experiment Results

Figure 8 presents the results of HMA detection of all the frameworks against the number of motes in the network. Figure 8(a) presents the overall HMA detection rate of the frameworks. The high variance in HMA detection confirms the randomness of HMA attacks. The HMA detection rate of LITMUS surpasses that of the other frameworks for any number of devices in the network. This high difference in HMA detection prowess stems from LITMUS's ability to handle collaborations with dishonest peers. The remaining frameworks never achieve a majority and fail to decide on

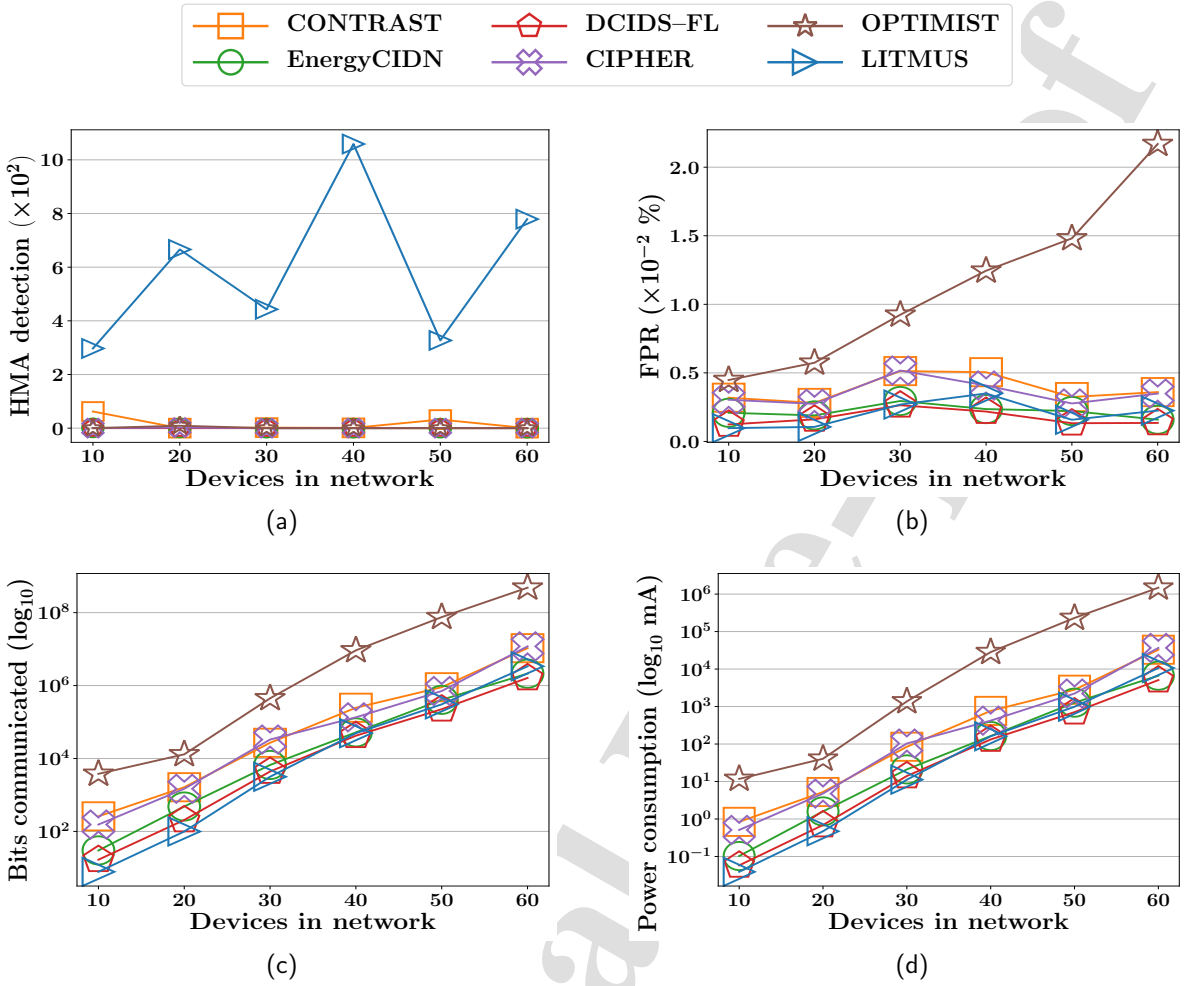


Figure 8: Results for HMA detection against network size. (a) HMA detection rate. (b) False positive rate. (c) Payload bits communicated. (d) Energy consumption in communication.

the maliciousness of attackers. Figure 8(b) presents the false positive rate (FPR) of the mechanisms. It clearly shows that OPTIMIST has the highest FPR, followed by CONTRAST. The FPR of LITMUS stays low, comparable with the rest of the frameworks. Figure 8(c) presents the payload bits communicated by the mechanisms in the log scale. It can be seen that OPTIMIST has the highest payload, followed by CONTRAST. LITMUS and DCIDS–FL incur the lowest communication overheads. Similarly, Figure 8(d) presents the energy consumed in communication, using the log scale, of the collaboration bits with a maximum packet size of 1500 bytes. It can be seen that OPTIMIST has the highest energy consumption, followed by CONTRAST. The energy consumption of LITMUS and DCIDS–FL is the lowest. Therefore, LITMUS performs better at overall HMA detection with the lowest overheads as compared to the other frameworks for different network sizes.

Figure 9 presents the results of HMA detection of the frameworks against the neighbor density of a mote. Figure 9(a) presents the HMA detection rates of the frameworks as a function of neighbor density. It can be seen that LITMUS outperforms all other state-of-the-art mechanisms, which struggle to detect HMAs irrespective of the availability of collaborators. Figure 9(b) presents the FPR of the mechanisms for all experiments. It is seen that the FPR of motes in LITMUS remains at $\approx 2\%$ and is comparable to that of the other mechanisms for all neighbor densities. Figure 9(c) presents the total number of bits the mechanisms communicate for all experiments for different neighbor densities. It is seen that DCIDS–FL has the fewest bits communicated during collaboration, and OPTIMIST has the most. LITMUS stays in between these ranges, comparable with all other frameworks. Similarly, Figure 9(d) presents the communication

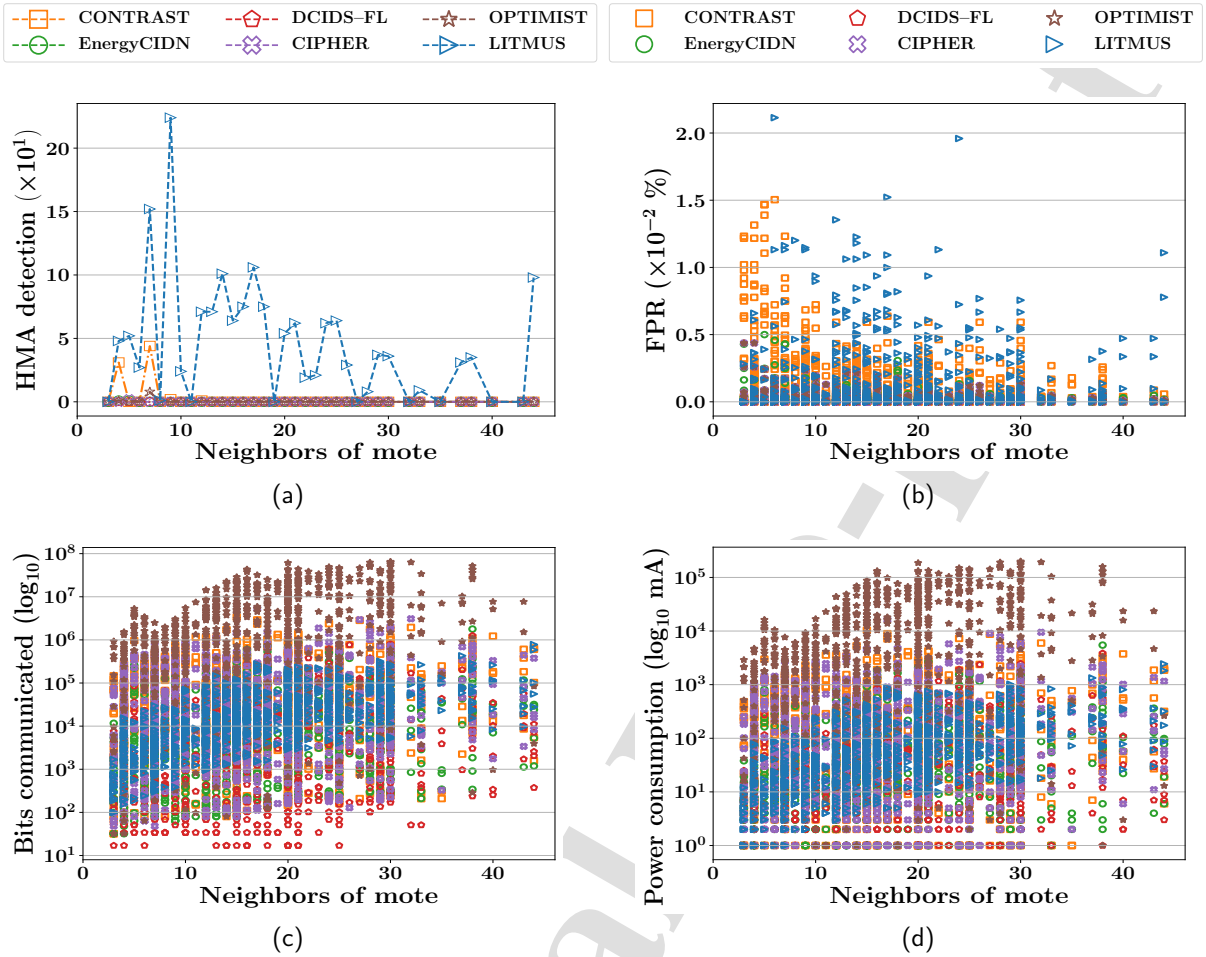


Figure 9: Results for HMA detection against mote neighbor density. (a) HMA detection rate. (b) False positive rate. (c) Payload bits communicated. (d) Energy consumption in communication.

energy consumption for the mechanisms under different neighbor densities. It is seen that DCIDS–FL has the lowest energy consumption during collaboration, and OPTIMIST has the highest. LITMUS stays in between these ranges, comparable with all other frameworks. Therefore, LITMUS outperforms other frameworks in HMA detection accuracy with lower overheads for different neighbor densities.

Figure 10 presents the contribution of collaboration as percentages of overall performance in the attack detection process plotted against different network sizes. Figure 10(a) presents the majority obtained by all mechanisms when LITMUS also used a majority-based scheme while decision-making. It shows that obtaining a majority is increasingly challenging for all frameworks as the number of neighbors increases in sparsely honest IIoT networks with a few honest peers. Thus, bloom filter-based aggregation schemes are better suited for sparsely honest networks. Figure 10(b) presents the true positive HMAs detected only by collaboration between peer devices. It clearly shows that state-of-the-art mechanisms struggle to confirm HMA attacks because they cannot obtain a majority in the aggregated responses across collaborations. Since LITMUS uses a bloom-filter-based, frequency-aware response-aggregation scheme, it can confirm HMA detections in sparsely honest networks, achieving high true-positive HMA detections. Figure 10(c) presents the percentage of anomalies detected by collaboration between peers relative to the overall HMA detection of the mechanisms. It is observed that state-of-the-art mechanisms achieve negligible benefits from peer collaboration for HMA detection. In contrast, LITMUS has a significant portion of its HMA detection supported by peer collaborations. Figures 10(b) and 10(c) show that, where majority-based aggregation schemes fail to make decisions in sparsely honest

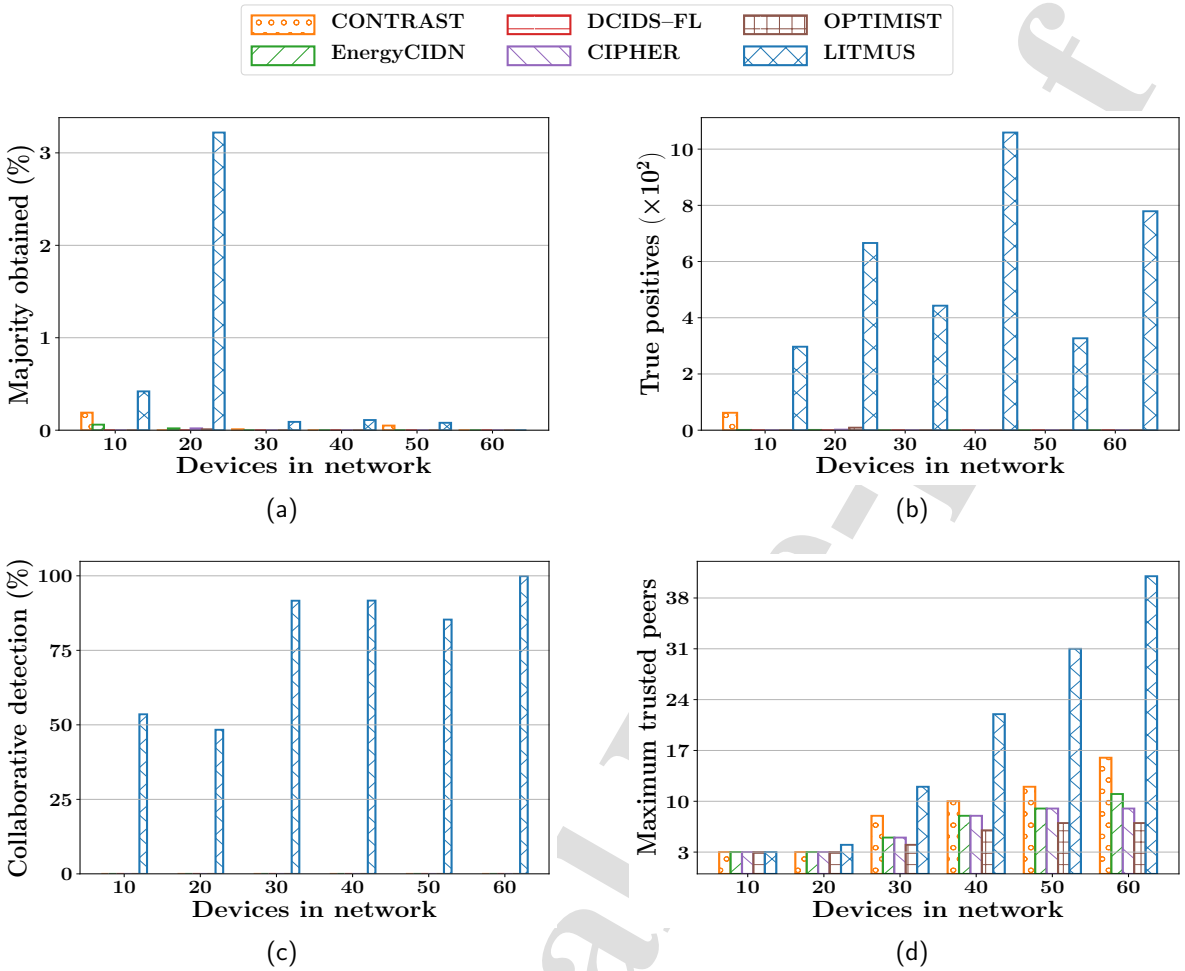


Figure 10: Benefits extracted from collaboration as percentages of overall performance in the attack detection process plotted against network size. (a) Majority obtained in collaboration. (b) True positive HMA detections by collaboration. (c) Percentage contribution of collaborations in HMA detection. (d) Maximum trusted collaborators during any iteration.

networks, bloom filter-based frequency-aware aggregation schemes can make decisions based on peer collaborations. Figure 10(d) presents the maximum number of trusted peers in a cycle that the mechanisms achieve in all experiments. Note that DCIDS–FL does not use a trust-based response aggregation scheme. Therefore, the mechanism is exempted from this analysis. Figure 10(d) shows that LITMUS has more trusted peers per cycle contributing to decision-making during attack detection than the state-of-the-art mechanisms. Thus, LITMUS can make better collaborative decisions than the state-of-the-art mechanisms.

Table 6 presents a summary of the benefits achieved by the frameworks through collaboration relative to their overall performance. It is seen that although LITMUS achieved the highest average majority in collaboration, it was still only $< 1\%$ and insufficient for making majority-based decisions. Additionally, LITMUS had a majority of its true positive detections supported by collaboration. The other mechanisms made negligible decisions based on collaboration. This high difference in HMA detection prowess stems from LITMUS's ability to handle collaborations with dishonest peers. The remaining frameworks never achieve a majority and fail to decide on the maliciousness of attackers. Consequently, the average number of anomalies detected by LITMUS was the highest among the frameworks, while that of the other mechanisms was negligible. Furthermore, LITMUS had the highest average number of trusted collaborators among all frameworks, enabling it to make better decisions during attack detection. The other frameworks had very few trusted

Table 6
Comparison of Performance based Only on Collaboration out of the Overall Performance of the Frameworks

Performance Parameter	CONTRAST	EnergyCIDN	DCIDS–FL	CIPHER	OPTIMIST	LITMUS
Majority obtained	0.042%	0.013%	0.002%	0.003%	0.002%	0.439%
True positive detections	0.02%	0.00%	0.00%	0.00%	0.00%	82.13%
Anomalies detected	16.17	0.17	0.67	1.50	3.00	639.79
Trusted peers in a cycle	8.7	6.5	0.0	6.16	4.5	19.5

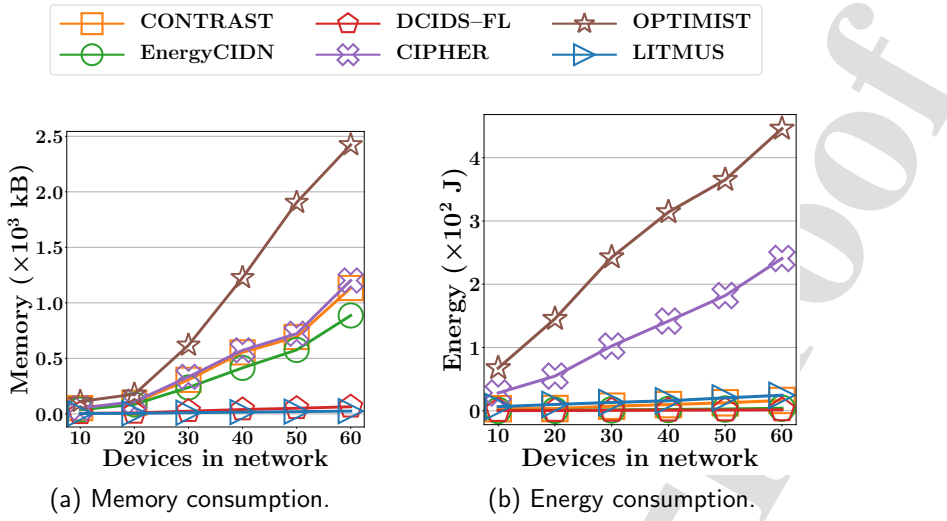
Table 7
Summary of the Average Experimental Outcomes of the Frameworks for the Deploy Phase of a Network With 60 Devices

Performance Parameter	CONTRAST	EnergyCIDN	DCIDS–FL	CIPHER	OPTIMIST	LITMUS
HMA detections	26 (2.89%)	11 (1.22%)	8 (0.89%)	31 (3.44%)	17 (1.89%)	779 (86.56%)
False positive rate	3.60%	1.59%	1.35%	3.49%	21.69%	2.24%
Bits communicated	10596594	2123742	1612881	11889778	4346584779	3375431
Energy overhead (J)	16.2	4.3	1.4	240.7	448.8	24.9
Memory overhead (MB)	1.13	0.89	0.06	1.21	2.42	0.03

collaborators. Since DCIDS–FL does not use a trust mechanism, it had no trusted collaborators. Thus, LITMUS achieved the best peer collaboration results compared to the state-of-the-art mechanisms.

Table 7 presents a summary of the experimental outcomes for the mechanisms during the deploy phase of a network with 60 devices. The attacker ratio was maintained at 10% of the total number of devices in the network for all experiments, as presented in Table 5. A maximum neighbor density of 40 devices per mote was used, as presented in Figure 9. All the neighbors were available for honest and dishonest collaboration. It is observed that LITMUS achieved the highest HMA detection rate, while DCIDS–FL achieved the lowest. This is attributed to the lack of a trust mechanism in DCIDS–FL for a network with dishonest collaborators. LITMUS uses a robust trust mechanism that provides it with immunity against various attacks and noise. The high difference in HMA detection prowess stems from LITMUS's ability to handle collaborations with dishonest peers. The remaining frameworks never achieve a majority and fail to decide on the maliciousness of attackers. Additionally, LITMUS achieved a low FPR when compared with frameworks with similar HMA detection accuracy. This is because LITMUS uses a Bloom filter-based response aggregation scheme, unlike the majority-based schemes of the other frameworks. Furthermore, LITMUS had the fewest bits communicated for collaboration among the other frameworks. This is again attributed to the use of Bloom filters for communication. It was observed that OPTIMIST used the maximum energy while DCIDS–FL used the minimum. LITMUS stayed between these two to provide a tradeoff between HMA detection accuracy and resource conservation. Lastly, LITMUS had the minimum memory overhead among the frameworks, followed by DCIDS–FL, while OPTIMIST had the highest. This is due to the use of Bloom filters for peer communication by LITMUS. Thus, LITMUS outperformed all other frameworks in HMA detection with a few honest collaborators while consuming fewer device resources.

Figure 11 shows the memory and energy consumptions of the frameworks during attacks. Figure 11(a) shows that OPTIMIST consumed the maximum memory while LITMUS and DCIDS–FL consumed the minimum. This is because the large detection model in OPTIMIST consumes significant memory, whereas LITMUS uses a small model that consumes very little memory. Similarly, Figure 11(b) shows that OPTIMIST consumed the maximum energy, closely followed by CIPHER, while the other frameworks remained comparable. This is because these two frameworks use resource-intensive neighbor monitoring approaches for trust establishment. LITMUS does not rely on such schemes, so it consumes less energy. Table 8 shows the model sizes of the frameworks and their execution times per cycle. OPTIMIST had the largest model size, followed by CONTRAST. EnergyCIDN had the smallest model size, closely followed by LITMUS. Even with a larger model size, LITMUS had a lower memory footprint than EnergyCIDN, as shown in Figure 11(a), due to its lightweight subschemes. Also, as shown in Table 8, OPTIMIST had the longest execution time per cycle, followed by CIPHER, while DCIDS–FL had the shortest. LITMUS remained between these ranges. Therefore, LITMUS maintained high HMA detection performance with comparable communication and execution overheads, and saved more energy than comparable state-of-the-art frameworks.



(a) Memory consumption. (b) Energy consumption.

Figure 11: Memory and energy consumption of the frameworks during attacks against network size.

Table 8
Comparison of the Total Model Sizes and Execution Times per Cycle for the Frameworks

Performance Metric	CONTRAST	EnergyCIDN	DCIDS–FL	CIPHER	OPTIMIST	LITMUS
Model Size (kB)	1.71	0.42	1.13	1.05	90.63	0.43
Execution Time (s)	0.42	0.14	0.04	13.18	13.66	0.61

Table 9
Sensitivity Analysis of LITMUS Against Different Values of Bloom filter Parameters for a Network with 60 Devices

Result	Bit Vector Size, m ($\times 10^1$ bits)					Hash Functions Used, k					False Positive Rate, q ($\times 10^{-3}$ %)				
Metric	8	11	14	17	20	4	6	7	9	10	0.5	1	5	10	50
ACC (%)	74.3	79.1	84.2	86.7	90.6	74.3	79.1	84.2	86.7	90.6	90.6	86.7	84.2	79.1	74.3
FPR (%)	2.8	2.5	2.2	2.1	1.9	2.8	2.5	2.2	2.1	1.9	1.9	2.1	2.2	2.5	2.8
CO (MB)	1.89	2.74	3.37	4.19	4.84	1.89	2.74	3.37	4.19	4.84	4.84	4.19	3.37	2.74	1.89

ACC = Accuracy, FPR = False Positive Rate, CO = Communication Overhead.

Table 9 presents the sensitivity of LITMUS towards different Bloom filter parameters, namely, the bit vector size (m), the number of hash functions used (k), and the FPR of the Bloom filter (q). Since all three parameters are interconnected, they affect each other directly as reflected in Table 9. It is observed that the framework's HMA detection accuracy increases with both the bit vector size and the number of hash functions. Also, HMA detection accuracy decreases as the Bloom filter's FPR increases. Consequently, the FPR of the framework decreases with increasing bit vector size and the number of hash functions used, and increases with the FPR of the Bloom filters. However, increasing the bit vector size and the number of hash functions used increases the framework's communication overhead. Also, decreasing the FPR of the Bloom filters increased the overall communication overhead. Thus, an increase in privacy offered by a larger bloom filter with more hash functions and a lower FPR results in higher accuracy, albeit at a higher communication overhead. Therefore, a trade-off between accuracy and privacy is desirable, based on the network requirements.

Figure 12 provides the performance of the trust module and attack resiliency of the different frameworks. As DCIDS–FL does not use a trust-based collaboration scheme, it does not provide any attack resiliency and has not been presented here. Figure 12(a) shows that LITMUS detects inconsistency attacks and penalizes the attacker incrementally for attack repetitions while also providing scope for behavioral improvement. The other frameworks do not provide similar provisions. Figure 12(b) shows that LITMUS detects and penalizes betrayal attacks, whereas the other frameworks cannot do so at an early stage of deployment. Figure 12(c) shows that LITMUS detects the maliciousness

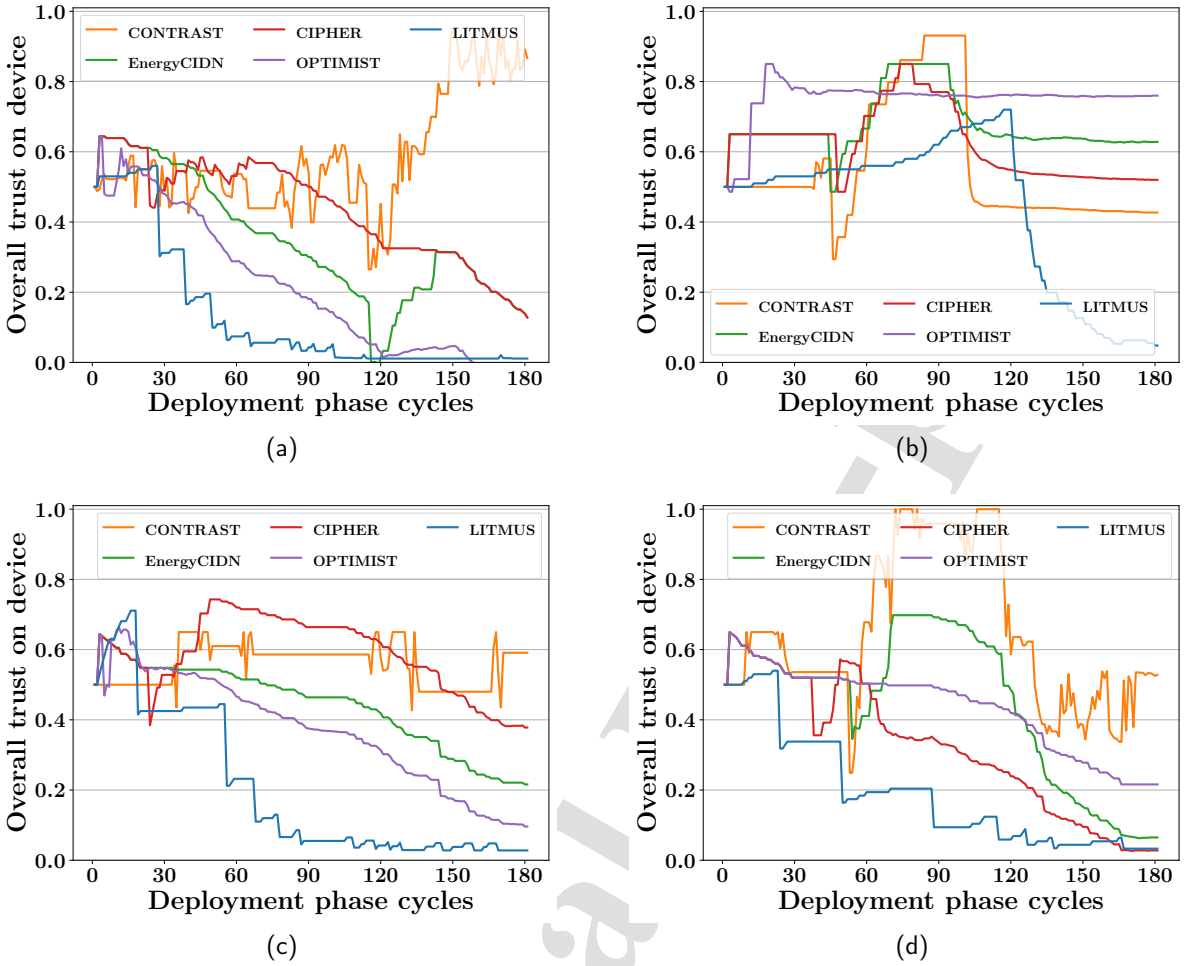


Figure 12: Attack resiliency of frameworks that use trust management. (a) Trust penalty for inconsistency attacks. (b) Trust penalty for betrayal attacks. (c) Trust penalty for newcomer attacks. (d) Trust penalty for mimicry attacks.

Table 10
Ablation Study of LITMUS to Determine the Contributions of its Subschemes for a Network with 60 Devices

Performance Parameter	HIDS Only	NIDS Only	No Bloom	No Trust	No Collab	Complete
HMA detections	94 (10.44%)	664 (73.78%)	429 (47.67%)	358 (39.78%)	487 (54.11%)	779 (86.56%)
False positive rate	0.41%	2.01%	8.40%	12.02%	10.59%	2.24%
Bits communicated	3375431	3375431	16921121	3375431	0	3375431
Energy overhead (J)	24.6	24.7	355.2	24.9	2.8	24.9
Memory overhead (MB)	0.02	0.02	0.81	0.02	0.01	0.02

of newcomer nodes early in the deployment phase and penalizes the attacker, while the other frameworks cannot do so at an early stage of deployment. Figure 12(d) shows that LITMUS detects mimicry attacks and penalizes the attacker, while the other frameworks cannot do so at an early stage of deployment. Therefore, the trust management in LITMUS is more robust to various attacks compared to the other frameworks.

Table 10 presents the ablation study of LITMUS for a network with 60 devices, to determine the contribution of its various subschemes. The framework, with all its subschemes (Complete), has been compared with five ablated

versions. The first one without the NIDS (HIDS Only), the second one without the HIDS (NIDS Only), the third one without the Bloom filter (No Bloom), the fourth one without the trust mechanism (No Trust), and the fifth one without any collaboration (No Collab). It is observed that the Complete version outperforms all its ablated versions in HMA detection accuracy. The HIDS Only and the NIDS Only versions outperform the Complete version in FPR. The No Collab version outperforms the Complete version in communication overhead. The HIDS Only, NIDS Only, and No Collab versions outperform the Complete version in terms of energy overhead. The No Collab version also outperforms the Complete version in memory overhead. Thus, each of its ablated versions performs better on some performance parameters while sacrificing others, and it also loses HMA detection accuracy. Therefore, the Complete version of LITMUS performs the best in collaborative HMA detection for networks with very few honest peers and conserves more device resources than the state-of-the-art mechanisms.

5. Conclusion

Hybrid implementations promise better detection of HMA footprints compared to pure NIDS or HIDS implementations. Collaborative IDSs further improve their attack detection accuracy by exchanging attack information with peers. However, most collaborative IDSs use majority-based schemes that detect attacks when a majority of truthful peers are available during decision-making. In addition, such solutions incur heavy communication overhead and do not address data privacy issues. Distributed collaborative IDS implementations reduce this communication overhead at the cost of reduced attack detection accuracy. To improve on the trade-off between attack detection accuracy and communication overhead while addressing data privacy concerns and making it possible to take decisions in collaborations without obtaining a majority, this work proposes a **L**ightweight d**I**stributed collabora**T**ive IDS for **M**imicry-based intr**U**sion detection in **S**parsely honest IIoT networks (LITMUS).

The proposed solution LITMUS uses a hybrid IDS scheme for HMA detection, a bloom filter-based communication scheme that ensures data privacy and reduces communication overhead, a collaboration scheme using frequency-aware response aggregation and resolution of data privacy concerns, and a trust management scheme that incorporates multiple trust factors to develop robust immunity against insider attacks and to avoid the use of resource-intensive neighbor monitoring approaches. Theoretical analysis and extensive simulations have shown that LITMUS outperforms the state of the art in both HMA detection accuracy and communication overhead while ensuring data privacy. It also reduces resource consumption by deploying frequency-aware collaboration in IIoT networks with only a few honest peers, which may not always form a majority.

It is noted that LITMUS runs on all host devices and consumes their resources, albeit minimally. In the future, the placement of the IDS in strategically optimal devices will be addressed to prolong network lifetime. Additionally, LITMUS assumes relatively stable networks with static neighborhoods for prolonged collaborative HMA detection. In the future, this research will be extended to include dynamic network conditions, such as network churn, clock synchronization issues, partial connectivity, and node mobility, to improve overall IDS protection.

CRediT authorship contribution statement

Surja Sanyal: Conceptualization of this study, Data curation, Methodology, Software, Visualization, Writing – original draft. **Manas Khatua:** Investigation, Supervision, Validation, Writing – review and editing, Project administration. **Pratik Chattopadhyay:** Investigation, Supervision, Validation, Writing – review and editing.

Data availability

The dataset used in this study is available at IEEE Dataport [57].

Declaration of competing interest

The authors declare that they have no known competing interests to report.

References

[1] Sihan Wang, Guan-Hua Tu, Xinyu Lei, Tian Xie, Chi-Yu Li, Po-Yi Chou, Fucheng Hsieh, Yiwen Hu, Li Xiao, and Chunyi Peng. Insecurity of operational cellular IoT service: new vulnerabilities, attacks, and countermeasures. In *Proc. of the 27th Annual International Conference on Mobile Computing and Networking (MobiCom)*, pages 437–450. ACM New York, NY, USA, 2021.

[2] Chao Liu, Boxi Chen, Wei Shao, Chris Zhang, Kelvin K. L. Wong, and Yi Zhang. Unraveling Attacks to Machine-Learning-Based IoT Systems: A Survey and the Open Libraries Behind Them. *IEEE Internet of Things Journal*, 11(11):19232–19255, 2024.

[3] Zhaowei Tan, Boyan Ding, Jinghao Zhao, Yunqi Guo, and Songwu Lu. Data-plane signaling in cellular IoT: attacks and defense. In *Proc. of the 27th Annual International Conference on Mobile Computing and Networking (MobiCom)*, pages 465–477. ACM New York, NY, USA, 2021.

[4] Gaetano Cimino and Vincenzo Deufemia. SIGFRID: Unsupervised, platform-agnostic interference detection in IoT automation rules. *ACM Transactions on Internet of Things*, 6(2):1–33, 2025.

[5] Devkishen Sisodia, Jun Li, Samuel Mergendahl, and Hasan Cam. A Two-Mode, Adaptive Security Framework for Smart Home Security Applications. *ACM Transactions on Internet of Things*, 5(2):1–31, 2024.

[6] Tianben Wang, Zhangben Li, Honghao Yan, Xiantao Liu, Boqin Liu, Shengjie Li, Zhongyu Ma, Jin Hu, Daqing Zhang, and Tao Gu. AudioGuard: Omnidirectional Indoor Intrusion Detection Using Audio Device. *ACM Transactions on Internet of Things*, 5(1):1–22, 2023.

[7] Bui Duc Son, Nguyen Tien Hoa, Trinh Van Chien, Waqas Khalid, Mohamed Amine Ferrag, Wan Choi, and Merouane Debbah. Adversarial Attacks and Defenses in 6G Network-Assisted IoT Systems. *IEEE Internet of Things Journal*, 11(11):19168–19187, 2024.

[8] Chetan Parampalli, R Sekar, and Rob Johnson. A practical mimicry attack against powerful system-call monitors. In *Proc. of the ACM symposium on Information, computer and communications security*, pages 156–167. ACM New York, NY, USA, 2008.

[9] Felix Erlacher and Falko Dressler. On High-Speed Flow-Based Intrusion Detection Using Snort-Compatible Signatures. *IEEE Transactions on Dependable and Secure Computing*, 19(1):495–506, 2022.

[10] Kushagra Agrawal, Tejasvi Alladi, Ayush Agrawal, Vinay Chamola, and Abderrahim Benslimane. NovelADS: A Novel Anomaly Detection System for Intra-Vehicular Networks. *IEEE Transactions on Intelligent Transportation Systems*, 23(11):22596–22606, 2022.

[11] Surja Sanyal, Manas Khatua, and Pratik Chattopadhyay. An Adaptive IDS Scheduling Framework for Host Based Mimicry Attack Detection in IoT Networks. In *2024 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS)*, pages 1–6. IEEE, 2024.

[12] Himanshi Babbar and Shalli Rani. FRHIDS: Federated Learning Recommender Hybrid Intrusion Detection System Model in Software-Defined Networking for Consumer Devices. *IEEE Transactions on Consumer Electronics*, 70(1):2492–2499, 2024.

[13] Taki Eddine Toufik Djaidja, Bouziane Brik, Sidi Mohammed Senouci, Abdelwahab Boualouache, and Yacine Ghamri-Doudane. Early Network Intrusion Detection Enabled by Attention Mechanisms and RNNs. *IEEE Transactions on Information Forensics and Security*, 19:7783–7793, 2024.

[14] Pradeepkumar Bhale, Debanjan Roy Chowdhury, Santosh Biswas, and Sukumar Nandi. OPTIMIST: lightweight and transparent IDS with optimum placement strategy to mitigate mixed-rate DDoS attacks in IoT networks. *IEEE Internet of Things Journal*, 10(10):8357–8370, 2023.

[15] Zia Ul Islam Nasir, Adnan Iqbal, and Hassaan Khaliq Qureshi. Securing cyber-physical systems: A decentralized framework for collaborative intrusion detection with privacy preservation. *IEEE Transactions on Industrial Cyber-Physical Systems*, 2:303–311, 2024.

[16] Junjiang He, Cong Tang, Wenshan Li, Tao Li, Li Chen, and Xiaolong Lan. BR-HIDF: An Anti-Sparsity and Effective Host Intrusion Detection Framework Based on Multi-Granularity Feature Extraction. *IEEE Transactions on Information Forensics and Security*, 19:485–499, 2024.

[17] Sunwoo Ahn, Hayoon Yi, Younghan Lee, Whoi Ree Ha, Giyeol Kim, and Yunheung Paek. Hawkware: network intrusion detection based on behavior analysis with ANNs on an IoT device. In *Proc. of 57th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2020.

[18] Hyunjun Kim, Sunwoo Ahn, Whoi Ree Ha, Hyunjae Kang, Dong Seong Kim, Huy Kang Kim, and Yunheung Paek. Panop: Mimicry-Resistant ANN-Based Distributed NIDS for IoT Networks. *IEEE Access*, 9:111853–111864, 2021.

[19] Aashma Uprety, Danda B Rawat, and Brian Sadler. Human Immune System Inspired Security for Federated Learning-Empowered Internet of Things. *ACM Transactions on Internet of Things*, 6(2):1–18, 2025.

[20] Ulya Sabeel, Shahram Shah Heydari, Khalil El-Khatib, and Khalid Elgazzar. Unknown, Atypical and Polymorphic Network Intrusion Detection: A Systematic Survey. *IEEE Transactions on Network and Service Management*, 21(1):1190–1212, 2023.

[21] Bing Gao, Bing Bu, Wei Zhang, and Xiang Li. An Intrusion Detection Method Based on Machine Learning and State Observer for Train-Ground Communication Systems. *IEEE Transactions on Intelligent Transportation Systems*, 23(7):6608–6620, 2022.

[22] Yunfan Huang, Maode Ma, Wong Jee Keen Raymond, and Chee-Onn Chow. An adaptive intrusion detection system for the internet of things using large language models and post-quantum-secure blockchain. *Computer Networks*, page 111819, 2025.

[23] Mengdie Huang, Hyunwoo Lee, Ashish Kundu, Xiaofeng Chen, Anand Mudgerikar, Ninghui Li, and Elisa Bertino. ARIoTEDef: Adversarially robust IoT early defense system based on self-evolution against multi-step attacks. *ACM Transactions on Internet of Things*, 5(3):1–34, 2024.

[24] Hakan Kayan, Ryan Heartfield, Omer Rana, Pete Burnap, and Charith Perera. CASPER: Context-aware IoT anomaly detection system for industrial robotic arms. *ACM Transactions on Internet of Things*, 5(3):1–36, 2024.

[25] Surja Sanyal, Manas Khatua, and Pratik Chattopadhyay. CIPHER: A Collaborative IDS Placement and Scheduling Framework for Host-Oriented Mimicry Attack Detection in IoT Networks. *IEEE Internet of Things Journal*, 2026. DOI: https://doi.org/10.1109/JIOT.2026.3689273.

[26] Aulia Arif Wardana and Parman Sukarno. Taxonomy and Survey of Collaborative Intrusion Detection System using Federated Learning. *ACM Computing Surveys*, 57(4):1–36, 2024.

[27] Emmanouil Vasilomanolakis, Shankar Karuppayah, Max Mühlhäuser, and Mathias Fischer. Taxonomy and survey of collaborative intrusion detection. *ACM computing surveys (CSUR)*, 47(4):1–33, 2015.

[28] Wenjuan Li, Weizhi Meng, and Lam For Kwok. Surveying trust-based collaborative intrusion detection: state-of-the-art, challenges and future directions. *IEEE Communications Surveys & Tutorials*, 24(1):280–305, 2021.

[29] Wenjuan Li, Philip Rosenberg, Mads Glisby, and Michael Han. Designing energy-aware collaborative intrusion detection in IoT networks. *Journal of Information Security and Applications*, 81:103713, 2024.

[30] Santiago de Diego, Cristina Regueiro, and Gabriel Macia-Fernandez. Collaborative credentials for the Internet of Things. *Computer Networks*, 251:110629, 2024.

[31] Michael E Locasto, Janak J Parekh, Salvatore Stolfo, Angelos D Keromytis, Tal G Malkin, and Vishal Misra. Collaborative distributed intrusion detection. *Columbia University Computer Science Technical Reports, CUCS-012-04*, 2004.

[32] Richeng Jin, Xiaofan He, and Huaiyu Dai. Collaborative IDS Configuration: A Two-Layer Game-Theoretic Approach. *IEEE Transactions on Cognitive Communications and Networking*, 4(4):803–815, 2018.

[33] Yair Meidan, Dan Avraham, Hanan Libhaber, and Asaf Shabtai. CADeSH: Collaborative Anomaly Detection for Smart Homes. *IEEE Internet of Things Journal*, 10(10):8514–8532, 2023.

[34] Pradeep Kannadiga and Mohammad Zulkernine. DIDMA: A distributed intrusion detection system using mobile agents. In *Proc. of Sixth International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing and First ACIS International Workshop on Self-Assembling Wireless Network*, pages 238–245. IEEE, 2005.

[35] Vinod Yegneswaran, Paul Barford, and Somesh Jha. Global intrusion detection in the domino overlay system. Technical report, University of Wisconsin-Madison Department of Computer Sciences, 2003.

[36] Bowen Hu, Chunjie Zhou, Yu-Chu Tian, Yuanqing Qin, and Xinjue Junping. A Collaborative Intrusion Detection Approach Using Blockchain for Multimicrogrid Systems. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 49(8):1720–1730, 2019.

[37] Wenjuan Li, Weizhi Meng, Javier Parra-Arnau, and Kim-Kwang Raymond Choo. Enhancing challenge-based collaborative intrusion detection against insider attacks using spatial correlation. In *Proc. of IEEE Conference on Dependable and Secure Computing (DSC)*, pages 1–8. IEEE, 2021.

[38] Wenjuan Li, Weizhi Meng, Lam-For Kwok, and HS Horace. Enhancing collaborative intrusion detection networks against insider attacks using supervised intrusion sensitivity-based trust management model. *Journal of Network and Computer Applications*, 77:135–145, 2017.

[39] Fang Liu, Xiuzhen Cheng, and Dechang Chen. Insider attacker detection in wireless sensor networks. In *Proc. of IEEE INFOCOM*, pages 1937–1945. IEEE, 2007.

[40] Carol J Fung, Jie Zhang, Issam Aib, and Raouf Boutaba. Robust and scalable trust management for collaborative intrusion detection. In *Proc. of IFIP/IEEE International Symposium on Integrated Network Management*, pages 33–40. IEEE, 2009.

[41] Hao-Jen Chien, Hossein Khalili, Amin Hass, and Nader Sehatbakhsh. Enc2: Privacy-Preserving Inference for Tiny IoTs via Encoding and Encryption. In *Proc. of the 29th Annual International Conference on Mobile Computing and Networking (MobiCom)*, pages 1–16. ACM New York, NY, USA, 2023.

[42] Dominic Lightbody, Duc-Minh Ngo, Andriy Temko, Colin Murphy, and Emanuel Popovici. Host-Based Intrusion Detection System for IoT using Convolutional Neural Networks. In *Proc. of 33rd Irish Signals and Systems Conference (ISSC)*, pages 1–7. IEEE, 2022.

[43] Juan E Tapiador and John A Clark. Masquerade mimicry attack detection: A randomised approach. *Computers & Security*, 30(5):297–310, 2011.

[44] Liyan Yang, Yubo Song, Shang Gao, Aiqun Hu, and Bin Xiao. Griffin: Real-Time Network Intrusion Detection System via Ensemble of Autoencoder in SDN. *IEEE Transactions on Network and Service Management*, 19(3):2269–2281, 2022.

[45] John T Hancock and Taghi M Khoshgoftaar. Survey on categorical data for neural networks. *Journal of Big Data*, 7(1):28, 2020.

[46] Walter Hugo Lopez Pinaya, Sandra Vieira, Rafael Garcia-Dias, and Andrea Mechelli. Autoencoders. In *Machine learning*, pages 193–208. Elsevier, 2020.

[47] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. In *Proc. of Network and Distributed System Security Symposium*, pages 1–15. Internet Society, 2018.

[48] Robert DiPietro and Gregory D Hager. Deep learning: RNNs and LSTM. In *Handbook of medical image computing and computer assisted intervention*, pages 503–519. Elsevier, 2020.

[49] Martin Riedmiller and A Lernen. Multi layer perceptron. *Machine Learning Lab Special Lecture, University of Freiburg*, 24, 2014.

[50] Sasu Tarkoma, Christian Esteve Rothenberg, and Eemil Lagerspetz. Theory and practice of bloom filters for distributed systems. *IEEE Communications Surveys & Tutorials*, 14(1):131–155, 2011.

[51] Andrei Broder and Michael Mitzenmacher. Network applications of bloom filters: A survey. *Internet Mathematics*, 1(4):485–509, 2004.

[52] Yuanhang Yang and Shuhui Chen. Multiple bloom filters. In *Proc. of VI International Conference on Network, Communication and Computing*, pages 59–63, 2017.

[53] Jeffrey Chen, Sehwan Hong, Warrick He, Jinyeong Moon, and Sang-Woo Jun. Eciton: Very Low-Power LSTM Neural Network Accelerator for Predictive Maintenance at the Edge. In *Proc. of 31st International Conference on Field-Programmable Logic and Applications (FPL)*, pages 1–8. IEEE, 2021.

[54] George Oikonomou, Simon Duquennoy, Atis Elsts, Joakim Eriksson, Yasuyuki Tanaka, and Nicolas Tsiftes. The Contiki-NG open source operating system for next generation IoT devices. *SoftwareX*, 18:101089–101097, 2022.

[55] Cedric Adjih, Emmanuel Baccelli, Eric Fleury, Gaetan Harter, Nathalie Mitton, Thomas Noel, Roger Pissard-Gibollet, Frederic Saint-Marcel, Guillaume Schreiner, Julien Vandaele, and Thomas Watteyne. FIT IoT-LAB: A large scale open experimental IoT testbed. In *2015 IEEE 2nd World Forum on Internet of Things (WF-IoT)*, pages 459–464. IEEE, 2015.

[56] Chris Sanders. *Practical packet analysis: Using Wireshark to solve real-world network problems*. No Starch Press, 2017.

[57] Surja Sanyal, Manas Khatua, and Pratik Chattopadhyay. IoT Dataset having Network and Host Features Implementing Mimicry Attack with Parent Switching. *IEEE Dataport*, 2025.

Surja Sanyal is pursuing his Ph.D. from the Departments of Computer Science and Engineering at the Indian Institute of Technology (IIT) Guwahati, India, and the Indian Institute of Technology (BHU) Varanasi, India. He received his M.Tech. in 2021 from the Computer Science and Engineering department of the National Institute of Technology (NIT) Durgapur, India. He was associated with Tata Consultancy Services (India) from 2011 to 2019 as a data analyst. His research interests include Machine Learning (ML) and Deep Learning (DL)- based security for Industrial Internet of Things (IIoT) networks, and the application of Game Theory (GT) in social settings.

Manas Khatua (Member, IEEE) received his Ph.D. degree from the Indian Institute of Technology (IIT) Kharagpur, Kharagpur, India, in 2016. He has been an assistant professor in the Department of Computer Science and Engineering at IIT Guwahati, India, since May 2018. Before that, he was an assistant professor at IIT Jodhpur, India, from 2016 to 2018, and a postdoctoral research fellow at the Singapore University of Technology and Design (SUTD), Singapore, from 2015 to 2016. He was associated with Tata Consultancy Services (India) from 2008 to 2010. His research interests include Performance Evaluation of Communication Protocols, Internet of Things, Sensor Networks, Network Security, and Cyber Physical Systems. He is the author of many publications in reputable international journals and conference proceedings. He is also a member of ACM.

Pratik Chattopadhyay (Member, IEEE) received his Ph.D. degree from the Indian Institute of Technology (IIT) Kharagpur, India, in 2015. He is currently an Associate Professor in the Department of Computer Science and Engineering at IIT (BHU), Varanasi, India. Prior to that, he was a Post-Doctoral Researcher at the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore. He received his M.E. degree in Computer Science and Engineering from Jadavpur University, India, in 2011, and his B.Tech. degree in Computer Science and Engineering from the Institute of Engineering and Management, Kolkata, India, in 2009. His research interests include machine learning, image processing, and data mining.

Declaration of interests

☒ The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

☐ The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: